



VILNIUS GEDIMINAS
TECHNICAL UNIVERSITY
FACULTY OF ELECTRONICS

DEPARTMENT OF ELECTRONIC SYSTEMS

Ozan Berk GULTEGIN

INVESTIGATION OF DEEP LEARNING METHODS FOR BEE BEHAVIOR RECOGNITION

Master's Thesis

Information Electronics Systems, State Code 6211BX015

Artificial Intelligence Systems

Informatics Engineering Study Area

VILNIUS GEDIMINAS TECHNICAL UNIVERSITY
FACULTY OF ELECTRONICS
DEPARTMENT OF ELECTRONIC SYSTEMS

CONFIRMS
Head of Department

(signature)
Prof Dr A. Serackis
_____, 2026

Ozan Berk GULTEGIN

**INVESTIGATION OF DEEP LEARNING METHODS FOR BEE BEHAVIOR
RECOGNITION**

GILIOJO MOKYMO METODŲ TYRIMAS BIČIŲ ELGSENAI ATPAŽINTI

Master Thesis

Information Electronics Systems, State Code 6211BX015

Artificial Intelligence Systems

Informatics Engineering Study Area

Supervisor

Assoc Prof Dr Tomyslav Sledevič

(scientific title and degree, name, surname) (signature) (date)

Vilnius, 2026

Investigation of Deep Learning Methods for Bee Behavior Recognition. Master's Thesis for Informatics Engineering degree. Vilnius Gediminas Technical University. Vilnius, 2026.

ABSTRACT

This thesis presents a two-stage computer vision pipeline for recognizing individual honeybee behaviors at the hive entrance. In the first stage, a YOLOv11-medium object detector augmented with SAHI tiling locates individual bees in high-resolution video frames, with ByteTracker assigning persistent identities across frames. In the second stage, a lightweight temporal Transformer classifier trained on frozen DINOv2-small features predicts behavior from a rolling 16-frame buffer of per-bee crops. Three behavior classes are recognized: fanning, trophallaxis, and neutral.

The detection model was trained on a purpose-tiled dataset of 30,001 images and achieves $\text{mAP}@0.5 = 0.992$ and $\text{mAP}@0.5:0.95 = 0.853$, a 31% relative improvement in localization quality over the prior YOLOv8-medium baseline. The behavior classifier was trained on a purpose-built action recognition dataset of temporally ordered per-bee sequences constructed using ByteTracker identity persistence, achieving macro $F1 = 0.96$ and overall accuracy of 0.97 on the validation set (1,829 samples). All three classes achieve $F1$ above 0.95, including trophallaxis, a social behavior not previously addressed in the bee monitoring literature.

The primary contribution of this work is the replacement of heuristic speed-based behavior inference with a supervised temporal classifier, enabling recognition of appearance-based social behaviors that motion statistics alone cannot identify. The pipeline is evaluated end-to-end on 44,680 frames across three source datasets (fanning: 18,000 frames; trophallaxis: 20,480 frames; detection: 6,200 frames), confirming correct integration of all stages.

Keywords: bee behavior recognition, YOLOv11, DINOv2, temporal Transformer, action recognition, SAHI, precision beekeeping

CONTENTS

1. Introduction	6
1.1 Relevance and Purpose of the Work	6
1.2 Tasks of the Work	6
1.3 Research and Analysis Methods Used	7
1.4 Novelty and Practical Benefit of the Work	8
1.5 The Structure of the Work	8
1.6 Chapter 1 Conclusion	9
2. Analytical literature Review	10
2.1 Automated Bee Monitoring	10
2.2 Object Detection – YOLO Family	11
2.3 Multi-Object Detection	12
2.4 Feature Extraction – Vision Transformers and DINOv2	13
2.5 Behavior Recognition Approaches	13
2.6 Limitations of Existing Solutions	14
2.7 Chapter 2 Conclusion	15
3. DEVELOPMENT of DEEP LEARNING METHOD FOR BEE BEHAVIOR RECOGNITION	16
3.1 System Overview	16
3.2 Stage 1 – Detection with YOLOv11 and SAHI	18
3.2.1 YOLOv11 – Medium	18
3.2.2 SAHI – Slicing Aided Hyper Inference	18
3.2.3 Motivation: Failure of Direct Resizing	19
3.3 Multi-Object Tracking — ByteTracker	19
3.4 Feature Extraction — DINOv2-small	20
3.5 Stage 2 – Temporal Sequence Classifier	21
3.5.1 Architecture	21
3.5.2 Training	22
3.6 End-to-End Integration	24
3.7 Chapter 3 Conclusion	24
4. Data Screening	25
4.1 Source Video Characteristics	25
4.2 Detection Dataset	26
4.3 Action Recognition Dataset	27
4.3.1 Behavior Classes	27
4.3.2 Dataset Construction	28
4.3.3 Dataset Statistics	29
4.4 Data Augmentation	31
4.4.1 Detection Augmentation	31
4.4.2 Action Recognition Augmentation	32
4.6 Chapter 4 Conclusion	33
5. Investigation of deep learning method for bee behavior recognition	34
5.1 Experimental Setup	34
5.2 Detection Results	34
5.2.1 Model selection	34
5.2.2 Final Model Performance	35
5.3 Action Recognition Results	36
5.3.1 Model Performance	36

5.3.2 Per-Class Results	37
5.3.3 Discussion	37
5.4 End-to-End Pipeline Evaluation	39
5.4.1 Detection Stage Reliability	40
5.4.2 Pipeline-level Classification Accuracy	40
5.4.3 Track Lifetime Analysis	43
5.5 Comparison to Prior Work	46
5.6 Chapter 5 Conclusion	47
6. Conclusions	48
6.1 Achievement of the Work Objective	48
6.2 What Should Be Done Differently	49
6.3 Tool Assessment	49
6.4 Summary	50
References	51
APPENDICES	53
Appendix A Source Code of the Temporal Sequence Classifier	53

1. INTRODUCTION

1.1 Relevance and Purpose of the Work

Honeybees play a critical role in global agriculture and ecosystems, contributing to the pollination of approximately one third of the world's food supply [1]. Monitoring the health and behavioral state of a bee colony is therefore of significant economic and ecological importance. Traditionally, hive inspection requires a beekeeper to physically open the hive a process that is time-consuming, stressful to the colony, and inherently subjective. The growing field of precision beekeeping seeks to address this limitation by providing continuous, non-invasive insight into colony conditions through sensors and automated analysis [2].

Visual monitoring of the hive entrance offers a particularly promising avenue for non-invasive assessment. The entrance is the primary interface between the colony and its environment: foraging activity, defensive responses, and social behaviors such as fanning and trophallaxis are all observable from outside the hive without disturbance. Automated recognition of these behaviors can give beekeepers early warning of anomalies declining activity, unusual clustering, or disrupted social interactions in real time.

Prior work at Vilnius Gediminas Technical University (VGTU) demonstrated that YOLOv8-based object detection can reliably locate and track individual bees on the hive landing board, and that behavioral patterns such as fanning and foraging can be inferred from motion analysis using heat maps and speed profiles [3, 4]. However, behavior identification in those works relied on hand-crafted heuristic rules rather than trained classifiers, limiting generalizability across different camera configurations and behavior types. Furthermore, trophallaxis the mouth-to-mouth transfer of food between bees, a key indicator of colony cohesion was not addressed.

The aim of this work is to develop and investigate a deep learning method for bee behavior recognition, realized as an automated two-stage computer vision pipeline combining a YOLOv11-based object detector with a temporal Vision Transformer classifier trained on labeled per-bee sequences.

1.2 Tasks of the Work

The aim of the thesis is to develop and investigate a deep learning method for bee behavior recognition. To achieve this aim, the following objectives are defined:

1. Review deep learning methods for behavior recognition.
2. Implement deep learning method for bee behavior recognition.
3. Investigate deep learning method for bee behavior recognition.

These objectives are carried out through the following specific tasks:

1. Review deep learning methods for object detection, multi-object tracking, visual feature extraction, and temporal behavior recognition in the context of bee monitoring.
2. Prepare and pre-process labeled datasets for detection model training and temporal action recognition, including image tiling and feature pre-computation.
3. Train and evaluate a YOLOv11-medium detection model with SAHI tiling for reliable small-bee localization in high-resolution video frames.
4. Design and train a temporal sequence classifier using frozen DINOv2 features for per-bee behavior recognition across three classes: fanning, trophallaxis, and neutral.
5. Integrate the detection and classification stages into a unified end-to-end pipeline with ByteTracker-based identity persistence.
6. Investigate and compare the performance of the proposed method against prior work, identify limitations, and propose directions for improvement.

1.3 Research and Analysis Methods Used

The work employs the following methods and tools:

- Deep learning for object detection: YOLOv11-medium, a single-stage anchor-free detector, is trained to localize individual bees in video frames.
- Slicing Aided Hyper Inference (SAHI): The input frame is divided into overlapping 640×640 tiles before detection, preserving the spatial resolution necessary to detect small bees in high-resolution footage.
- Multi-object tracking: ByteTrack assigns persistent identities to detected bees across frames, enabling the construction of per-bee temporal sequences.
- Self-supervised visual feature extraction: DINOv2-small, a frozen Vision Transformer pre-trained through self-distillation on a large unlabeled dataset, produces 768-dimensional feature vectors (CLS token concatenated with mean patch token) from each per-bee crop.
- Temporal sequence classification: A lightweight three-layer Transformer encoder classifies 16-frame feature sequences into behavior categories.

- Data augmentation: Geometric and photometric augmentation (including mosaic and copy-paste) for detection training; feature-space augmentation (Gaussian noise, temporal reversal, temporal jitter) for classifier training.
- Quantitative evaluation: Model performance is assessed using precision, recall, $mAP@0.5$, and $mAP@0.5:0.95$ for detection; per-class and macro-average F1-score for behavior classification.

1.4 Novelty and Practical Benefit of the Work

The primary novelty of this work is the replacement of heuristic, rule-based behavior inference with a supervised temporal classifier trained on labeled per-bee sequences. Prior systems at VGTU inferred behavior from speed thresholds and heat maps, approaches that are sensitive to camera calibration and frame rate, and that cannot distinguish behaviors with similar motion profiles. The learned classifier introduced here generalizes across recording conditions and extends the recognizable behavior vocabulary to include trophallaxis, a social behavior that has no distinguishing motion signature but a distinctive visual appearance. To the best of the author's knowledge, individual-level trophallaxis recognition from video has not been previously demonstrated in the bee monitoring literature.

A secondary contribution is the upgrade of the detection backbone from YOLOv8-medium to YOLOv11-medium combined with SAHI tiling, yielding a 31% relative improvement in $mAP@0.5:0.95$ over the prior baseline without sacrificing detection speed.

The practical value of the work lies in its application to precision beekeeping: automated, continuous recognition of fanning and trophallaxis at the hive entrance provides beekeepers with objective, real-time indicators of colony ventilation state and social cohesion, two signals relevant to early detection of hive stress, disease onset, or population decline without any physical disturbance to the colony.

1.5 The Structure of the Work

The thesis consists of five main chapters followed by conclusions, a reference list, and appendices.

Chapter 2 - Analytical Literature Review: Surveys existing methods for automated bee monitoring, the YOLO family of object detectors, multi-object tracking, and behavior recognition approaches. It concludes with an analytical assessment of why existing solutions are insufficient for the tasks addressed in this thesis.

Chapter 3 - Development of Deep Learning Method for Bee Behavior Recognition: Describes the architecture of the proposed two-stage pipeline in detail, covering the detection stage, tracking, feature extraction, and temporal classifier design. Mathematical formulations of key operations are provided.

Chapter 4 - Data Screening: documents the construction of both datasets: the 30,001-image detection dataset and the 104,888-image action recognition dataset. It addresses source video characteristics, collection criteria, temporal sequence structure, class imbalance, and augmentation strategies.

Chapter 5 - Investigation of Deep Learning Method for Bee Behavior Recognition: Presents detection and action recognition results, ablation studies across buffer sizes, and a comparison against prior VGTU work. End-to-end pipeline performance is evaluated and discussed.

Chapter 6 - Summary. Conclusions: Summarizes the achievements of the work, assesses task completion, identifies limitations, and proposes directions for future research.

1.6 Chapter 1 Conclusion

Automated behavior recognition at the hive entrance is a relevant and underexplored problem. Prior work at VGTU established that individual bee detection and tracking is feasible using YOLOv8, but behavior inference remained rule-based and limited to coarse motion patterns. This thesis addresses that gap with a fully learned, temporally aware pipeline. The following chapters detail the literature context, system design, dataset construction, and experimental evaluation.

2. ANALYTICAL LITERATURE REVIEW

This chapter surveys existing methods relevant to the two-stage pipeline developed in this thesis. It covers automated hive monitoring approaches, the evolution of the YOLO detection family, multi-object tracking, and behavior recognition methods. The chapter concludes by identifying the specific limitations of prior work that motivate the present approach. This chapter is primarily relevant to readers seeking to understand the context and design decisions behind the proposed system.

2.1 Automated Bee Monitoring

Monitoring bee colony health has been approached through multiple sensing modalities. Acoustic monitoring analyzing the sounds produced inside the hive has been used to detect the presence of a queen, identify swarming behavior, and estimate colony population [5, 6]. Thermal imaging has been applied to assess brood temperature and hive population density [7]. Weight sensors mounted beneath hives can track daily foraging activity and detect sudden colony loss [22]. While these modalities provide valuable aggregate colony-level signals, they offer limited ability to distinguish individual bee behaviors or detect fine-grained social interactions.

Visual monitoring at the hive entrance provides a complementary, behavior-specific signal. Cameras mounted above the landing board can capture individual bees continuously and non-invasively. The main challenge is the density and speed of bee activity during peak foraging, hundreds of bees can be present simultaneously, making individual detection and tracking computationally demanding.

Table 2.1. Comparison of hive monitoring modalities.

Modality	What it measures	Individual Level	Behavior-specific	Example
Acoustic	Sounds inside hive	No	Partial (swarming, queen)	Várkonyi et al. [5]
Thermal	Temperature, population density	No	No	Shaw et al. [7]
Weight sensors	Foraging activity, colony loss	No	No	Meikle & Holst [22]
Visual – Rule Based	Speed, trajectory, heat maps	Yes	Limited (Fanning, Foraging)	Sledevič et al. [3, 4]
Visual – Detection Based	Object detection, occurrence density	Yes	Limited (Fanning, Foraging)	Sledevič et al. [16]; Vdoviak et al. [19]

Visual – Learned	Appearance, temporal patterns	Yes	Fanning, Trophallaxis	This work
---------------------	----------------------------------	-----	--------------------------	-----------

Within visual monitoring, the last two years have seen a notable increase in scope and scale. Sledevič et al. [16] presented a comprehensive behavior recognition framework employing YOLOv8 for detection and instance segmentation at the hive entrance, processing footage from eight distinct hives to recognize foraging, fanning, washboarding, and defensive behaviors. The system achieved 98% mean detection accuracy at 50 fps, demonstrating the discriminative power of spatial trajectory analysis and occurrence density maps for multi-behavior identification. A subsequent study by Vdoviak et al. [19] evaluated a broader range of deep learning architectures for multi-class insect detection at the hive entrance including worker bees, pollen-bearing foragers, drones, and wasps with a focus on deployment on the NVIDIA Jetson AGX Orin embedded platform, extending visual monitoring to resource-constrained field installations.

Individual-level tracking has been approached using deep learning in combination with classical state estimation. Kongsilp et al. [20] demonstrated per-bee identity maintenance using Mask R-CNN for instance segmentation combined with a Kalman filter for trajectory estimation, achieving 77.5% MOTA on in-hive footage. A broader review of machine learning approaches to bee recognition and tracking [21] identified the main open challenges as occlusion handling, high-density activity, and the lack of large-scale annotated datasets limitations that remain relevant to hive-entrance monitoring.

2.2 Object Detection – YOLO Family

The YOLO (You Only Look Once) family of single-stage object detectors has become the dominant approach for real-time object detection since its introduction [8]. Unlike two-stage detectors that first propose candidate regions and then classify them, YOLO processes the entire image in a single forward pass, enabling detection at video frame rates.

YOLOv8, the version used in the prior VGTU works; introduced anchor-free detection heads and a revised backbone, achieving strong performance across detection, segmentation, and tracking tasks [9]. YOLOv11, the architecture used in this thesis, continues this trajectory with further architectural refinements to the C3k2 block structure and attention mechanisms, improving small-object detection performance at comparable inference speeds.

Sledevič and Plonis [3] compared YOLOv8 nano, small, and medium variants for bee detection, finding that YOLOv8-medium achieved the best trade-off between accuracy and speed, with $\text{mAP}@0.5 = 0.97$ and $\text{mAP}@0.5:0.95 = 0.65$. The medium-scale model was therefore selected as the basis for this work as well, upgraded to YOLOv11-medium. Subsequent work by Sledevič [17] directly compared YOLOv8, YOLO11, and YOLO12 on a dedicated 18,000-frame fanning dataset

comprising 57,597 annotated instances, evaluating both standard RGB inputs and two motion-enhanced input representations. This provides a direct benchmark for single-behavior detection methods in the bee domain.

Table 2.2. Selected milestones of the YOLO detector family.

Version	Key Architectural Change	mAp@0.5 on bee detection
YOLOv5	Anchor-based, CSP backbone	-
YOLOv8	Anchor-free detection head	0.97% (Sledevič et al. [3])
YOLOv11	C3k2 blocks, improved attention	0.992% (this work)

A practical challenge in bee detection is the small size of individual bees relative to the full frame. Standard object detectors trained at 640×640 resolution may miss small objects that occupy only a few pixels. SAHI (Slicing Aided Hyper Inference) [10] addresses this by dividing the input image into overlapping tiles at the native detector resolution, running detection on each tile independently, and merging the results. This approach has been shown to significantly improve recall for small objects without retraining the detector.

2.3 Multi-Object Detection

Multi-object tracking (MOT) assigns persistent identities to detected objects across video frames, enabling the construction of individual trajectories. For behavior recognition, persistent identity is essential, the classifier must accumulate a sequence of features from the same physical bee rather than from an arbitrary mixture of detections.

ByteTrack [11] is a high-performance tracker that associates nearly every detection box with a tracked identity, rather than discarding low-confidence detections as many prior methods do. It maintains two sets of tracks confirmed (high-confidence) and tentative (low-confidence), and performs IoU-based matching in two rounds, recovering occluded or temporarily low-confidence bees that would otherwise generate spurious new track IDs.

The prior VGTU works employed ByteTrack for bee tracking and reported that high-density activity, occlusions, and partial overlaps between bees caused the tracker to occasionally lose or switch identities [3, 4]. This is an inherent limitation of appearance-free IoU-based tracking and remains relevant in the current work.

2.4 Feature Extraction – Vision Transformers and DINOv2

Vision Transformers (ViTs) [12] apply the self-attention mechanism to image patches, learning long-range spatial dependencies that convolutional networks handle less naturally. Unlike convolutional backbones that process features hierarchically through local receptive fields, ViT processes all patches simultaneously, with global context available at every layer.

DINOv2 [13] is a self-supervised ViT trained on a large curated dataset (LVD-142M images) using self-distillation with no labels. DINOv2's CLS token, the global summary token prepended to the patch sequence; has been shown to encode rich, semantically meaningful visual features that transfer well to downstream tasks with minimal fine-tuning. In this thesis, the CLS token is concatenated with the mean of the 256 spatial patch tokens to form a 768-dimensional feature vector per frame, as described in Section 3.4. These properties make it well-suited as a frozen feature extractor; the network need not be trained on low level bee data to produce discriminative features for classifying bee behaviors.

2.5 Behavior Recognition Approaches

Prior work on bee behavior recognition falls into two broad categories.

- Rule-based approaches: Infer behavior from geometric or temporal statistics of tracked trajectories. Sledevič and Abromavičius [4] used per-bee speed profiles, computed with a 7-frame moving average filter at 50 fps to distinguish foraging (high speed, short dwell time), fanning (near-stationary, long dwell time), and washboarding (rhythmic oscillation). While interpretable, these methods are sensitive to camera calibration, frame rate, and recording conditions: a speed threshold of 10 pixels/s is meaningless across different setups unless pixel-to-millimeter calibration is performed.
- Learning-based approaches: For bee behavior are comparatively rare. Most published work either classifies behaviors at the colony level (aggregate activity) rather than the individual level or relies on manually engineered features. Individual-level behavior classification from video sequences treating each tracked bee as an independent classification instance has not been addressed in the bee monitoring literature prior to this work.

Recent work has begun to close this gap for specific behavior classes. The behavior of trophallaxis, mouth-to-mouth food transfer between bees; has received dedicated attention: Vdoviak and Sledevič [18] introduced a large-scale annotated trophallaxis dataset and evaluated YOLO-based architectures augmented with temporal encoding strategies injected as RGB input channels. The best configuration

(Temporal-YOLOv8 with 1-second motion averaging) achieved $\text{mAP}@0.5 = 0.955$, a 10.8-percentage-point improvement over the standard RGB baseline. This detection-based approach treats trophallaxis as an object category localized in the frame, in contrast to the classification-based approach of the present thesis, which assigns a behavior label to a tracked individual bee rather than to a spatial region. Fanning detection has been evaluated as a dedicated task by Sledevič [17], who compared YOLOv8, YOLO11, and YOLO12 across RGB and motion-enhanced inputs on an 18,000-frame dataset. The most directly comparable prior work in overall scope is Sledevič et al. [16], which combined YOLOv8 detection with trajectory analysis to recognize four behavior classes simultaneously; however, behavior inference in that work remained heuristic based on trajectory statistics and spatial density maps rather than a trained classifier and thus sharing the generalizability limitation described in Section 2.6.

In the broader action recognition literature, temporal models have demonstrated that short video clips contain substantially more discriminative information than individual frames [14]. The approach adopted in this thesis, extracting per-frame features with a frozen backbone and classifying the resulting feature sequence with a lightweight Transformer follows this paradigm while remaining tractable for small labeled datasets.

2.6 Limitations of Existing Solutions

The prior VGTV works [3, 4, 16] establish that individual bee detection and tracking at the hive entrance is feasible, and recent work [17, 18, 19] has begun to explore behavior-specific detection methods. However, three limitations motivate the present approach:

- 1- Heuristic behavior inference: Speed thresholds and occurrence density maps are not learned from labeled data and do not generalize across camera setups. They also cannot distinguish behaviors with similar motion profiles. The detection-based approach of [16] shares this limitation in its inference stage.
- 2- No individual-level classifier for social behaviors: Trophallaxis involves two or more bees in sustained contact. Detection-based approaches [18] localize the interaction region but do not associate the behavior with a specific tracked individual. A per-bee temporal classifier is necessary for individual-level attribution.
- 3- Coarse behavior vocabulary: Behaviors addressed in prior work are defined by large-scale motion patterns. Fine-grained social behaviors requiring per-bee appearance classification across time remain unaddressed outside this thesis.

The system developed in this thesis directly addresses the first two limitations and partially the third: a trained temporal classifier is demonstrated for fanning, trophallaxis, and a neutral baseline class, with behavior labels assigned per tracked individual.

2.7 Chapter 2 Conclusion

Automated bee monitoring has been approached through acoustic, thermal, and visual sensing. Visual monitoring at the hive entrance using the YOLO detection family has proven effective for locating individual bees, and recent work has demonstrated detection-based approaches for specific behaviors including fanning [17] and trophallaxis [18]. ByteTrack provides adequate tracking for moderate-density activity. Existing behavior recognition in this domain relies either on heuristic speed analysis or detection-based localization of interaction regions approaches that do not assign behavior labels to individually tracked bees and cannot generalize across camera setups. A learned temporal classifier operating on per-bee image features addresses these limitations and represents the primary novelty of the present work.

3. DEVELOPMENT OF DEEP LEARNING METHOD FOR BEE BEHAVIOR RECOGNITION

This chapter describes the architecture and design of the proposed two-stage behavior recognition pipeline. It is intended for readers who need to understand the technical implementation in sufficient depth to reproduce or extend the system. The chapter covers each stage in sequence detection, tracking, feature extraction, and classification and concludes with the integration strategy for end-to-end inference.

3.1 System Overview

The pipeline takes a sequence of video frames as input and produces, for each frame, a set of bounding boxes with associated behavior labels one per tracked bee. The processing stages are as follows:

- 1- Detection: Each frame is divided into overlapping 640×640 tiles by SAHI and passed through YOLOv11-medium, producing bounding boxes for all visible bees.
- 2- Tracking: ByteTracker associates detections across frames, assigning each bee a persistent track ID.
- 3- Feature extraction: For each tracked bee, the crop defined by its bounding box is resized to 224×224 pixels and passed through the frozen DINOv2-small backbone, producing a 768-dimensional feature vector formed by concatenating the global CLS token (384 dimensions) with the mean of the 256 spatial patch tokens (384 dimensions).
- 4- Temporal buffering: Each track maintains a rolling deque of the 16 most recent feature vectors.
- 5- Classification: Once a track has accumulated at least 4 frames, the feature buffer is passed to the Temporal Sequence Classifier, which outputs a softmax distribution over three classes: fanning, neutral, and trophallaxis

The design deliberately separates detection and classification into independently trainable modules. This allows each stage to be trained on purpose-built datasets and updated independently without retraining the other.

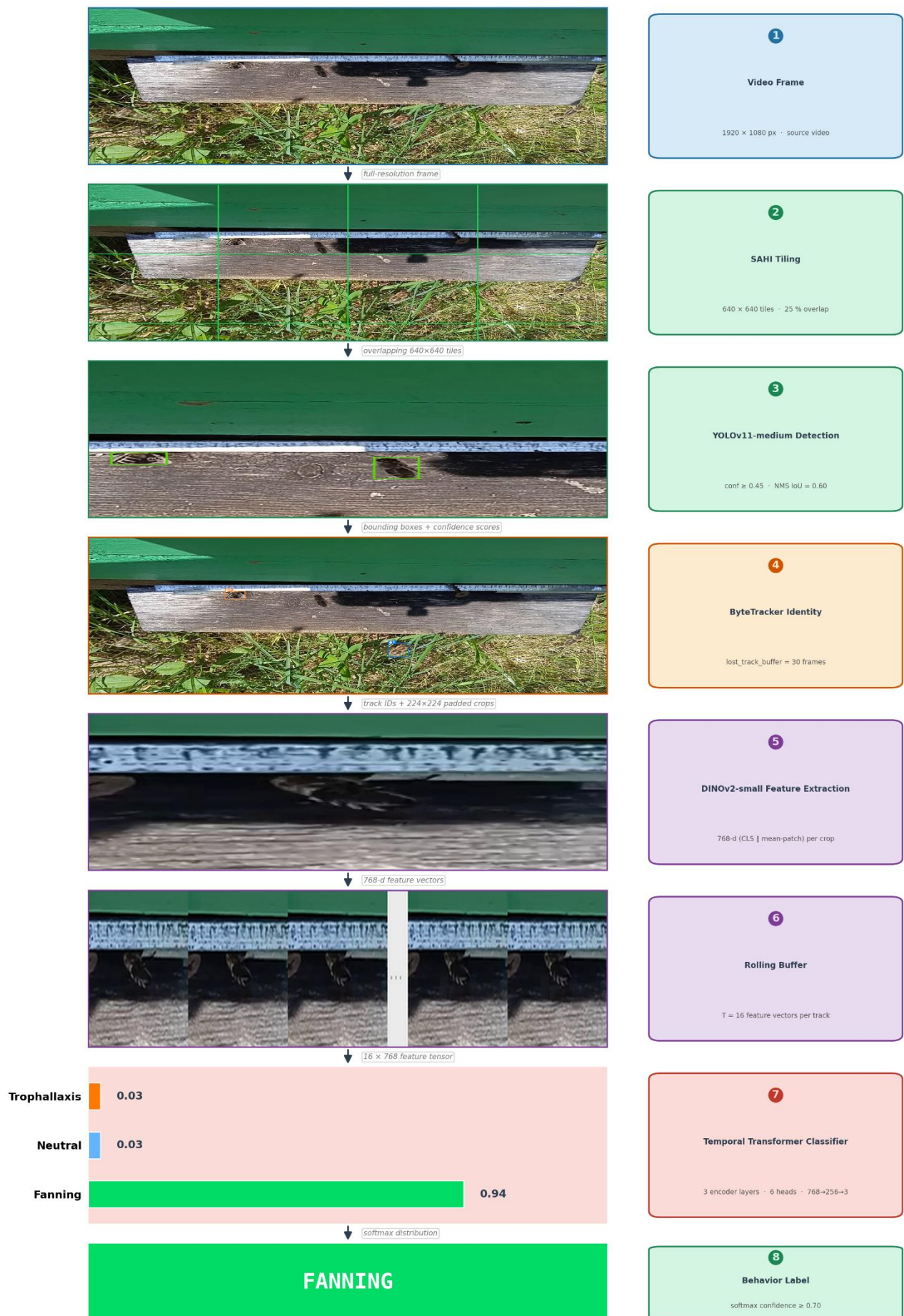


Fig. 3.1. Pipeline flow diagram

3.2 Stage 1 – Detection with YOLOv11 and SAHI

3.2.1 YOLOv11 – Medium

YOLOv11-medium is a single-stage anchor-free object detector. It processes an input image through a backbone network that produces multi-scale feature maps, followed by a feature pyramid neck that aggregates spatial information across scales, and detection heads that predict bounding boxes and class probabilities for each spatial location. The backbone uses C3k2 blocks, which replace the C2f bottleneck from YOLOv8 with a dual-kernel convolution design that captures multi-scale features more effectively particularly beneficial for small-object detection where feature richness at fine spatial scales is critical.

The model was initialized from ImageNet-pretrained weights and fine-tuned on the bee detection dataset (Section 4.1). Training was conducted with the AdamW optimizer with an initial learning rate of 0.0008, cosine learning rate decay, and a batch size of 6. An effective batch size of 24 was achieved through gradient accumulation (factor of 4). The model was trained for 238 epochs with early stopping patience of 50 epochs. Detection is performed at a confidence threshold of 0.45 detections with lower softmax confidence are discarded. This threshold was selected empirically to balance precision and recall on the validation set.

3.2.2 SAHI – Slicing Aided Hyper Inference

Individual bees occupy a small fraction of the total frame area in typical hive-entrance recordings. When a high-resolution frame is resized to the 640×640 resolution expected by the detector, small bees may be reduced to only a few pixels below the spatial resolution at which the network can reliably detect them.

SAHI addresses this by slicing the input frame into overlapping tiles of 640×640 pixels before detection and merging the resulting predictions with NMS (non-maximum suppression). Each tile is processed at the detector's native resolution, preserving the spatial detail necessary for detecting small objects. Tiles are generated with a 25% overlap in both dimensions to prevent bees near tile boundaries from being missed.

Formally, given a frame of size H×W, SAHI generates tiles at positions:

$$tile_{i,j} = crop(x_j, y_i, x_j + 640, y_i + 640), \quad (1)$$

where,

$$x_j = j \cdot (1 - 0.25) \cdot 640 \text{ and } y_i = i \cdot (1 - 0.25) \cdot 640 \quad (2)$$

Coordinates are clipped to the frame boundary and predictions from all tiles are merged using IoU-based NMS with threshold 0.6.



Figure 3.2. SAHI tiling illustration

3.2.3 Motivation: Failure of Direct Resizing

The SAHI tiling strategy was adopted in response to a concrete failure of the simpler alternative. An initial YOLOv11-medium model (v1) was trained with frames directly resized to 640×640 the detector's native input resolution without tiling. After 450 epochs this model achieved strong validation metrics: Precision = 0.978, Recall = 0.965, mAP@0.5 = 0.985, mAP@0.5:0.95 = 0.790. Despite these figures, the model performed poorly during inference: many bees were missed entirely, structural features of the hive (surface dents, textured areas) were flagged as bees, and similarly colored background regions such as yellowish grass were detected as bees with high confidence. Post-hoc analysis identified the root cause: downscaling a 1920×1080 frame to 640×640 reduces the pixel footprint of a typical bee to only a few pixels in the input space. At this scale, individual bees lose their defining visual characteristics and become indistinguishable from background texture variations. The model had learned to exploit positional and color correlations specific to the training set rather than the true appearance of a bee.

This result illustrates a known failure mode in small-object detection: validation metrics computed on in-distribution images can be misleadingly high when the model has memorized dataset-specific cues. The tiling approach described in Section 3.2.2 resolves this by presenting each bee at its native scale to the detector, eliminating the resolution bottleneck entirely.

3.3 Multi-Object Tracking — ByteTracker

ByteTracker maintains a set of active tracks, each characterized by a bounding box state estimated with a Kalman filter and a confidence score. At each frame, it performs detection-to-track association in two rounds:

High-confidence round: Detections with confidence ≥ 0.45 are matched to existing tracks using IoU distance.

Low-confidence round: Remaining unmatched detections (confidence < 0.45) are matched against unconfirmed tracks, recovering bees that temporarily produced weak detections due to occlusion or motion blur. Tracks not matched for more than 30 consecutive frames are discarded. This 30-frame buffer allows the tracker to maintain a bee's identity across brief occlusions for example, when one bee passes in front of another without creating spurious new identities.

Each track ID corresponds to a unique per-bee feature buffer in the classification stage. When a track is discarded, its buffer is cleared.

3.4 Feature Extraction — DINOv2-small

For each tracked bee at each frame, a crop is extracted from the frame using the bounding box coordinates with 35% proportional padding added on all sides. The padding ensures that the bee's body context (adjacent bees, substrate texture) is visible to the feature extractor, which is informative for trophallaxis detection where two bees are in close contact. Padding also provides positional context substrate texture, surrounding bees that helps the model distinguish fanning, which occurs near the hive entrance in a characteristic spatial setting, from visually similar stationary postures elsewhere on the landing board. The padded crop is resized to 224×224 pixels and normalized with the ImageNet mean and standard deviation:

$$x_{norm} = (x - \mu)/\sigma, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225], \quad (3)$$

The normalized crop is passed through the frozen DINOv2-small backbone. The output of interest is not the CLS token alone, but the concatenation of two complementary representations from the final transformer layer: the CLS token (384 dimensions), which encodes a global semantic summary of the entire crop, and the mean of the 256 spatial patch tokens (384 dimensions), which encodes spatially aggregated local texture and pose information. Concatenating these yields a 768-dimensional feature vector per frame. Including patch token statistics is particularly valuable for bee behavior: fanning produces wing-blur textures distributed across the bee's body, while trophallaxis involves a distinctive spatial arrangement of two bees in close antennae contact patterns that the spatially resolved patch mean captures better than the global CLS token alone.

The backbone weights are fully frozen during all training stages. This prevents overfitting: the effective number of trainable parameters is limited to the classifier only (approximately 14 million), while the 21.7-million-parameter backbone provides a stable, high-quality feature basis. The small variant was chosen for inference efficiency each tracked bee generates one feature vector per frame, so a lighter backbone reduces per-frame latency and memory footprint. DINOv2-base and DINOv2-large were not evaluated.

3.5 Stage 2 – Temporal Sequence Classifier

3.5.1 Architecture

The Temporal Sequence Classifier takes a sequence of $T = 16$ DINOv2 feature vectors as input one vector per buffered frame and outputs a probability distribution over three behavior classes. Its architecture is as follows:

- Input: $X \in \mathbb{R}^{B \times T \times 768}$ where B is the batch size and $T = 16$. Each frame's feature vector is the 768-dimensional DINOv2 representation (CLS || mean-patch, see Section 3.4).
- CLS token prepension: A learnable classification token $c \in \mathbb{R}^{1 \times 768}$ is prepended to the sequence, following the ViT convention:

$$X' = [c \parallel x_1 \parallel x_2 \parallel \dots \parallel x_T] \in \mathbb{R}^{B \times (T+1) \times 768} \quad (4)$$

- Positional Embedding: Learnable position embeddings $P \in \mathbb{R}^{(T+1) \times 768}$ are added element-wise to X' , encoding the temporal position of each frame.
- Transformer encoder: Three layers of pre-norm Transformer encoder blocks [15] process the sequence. Each block applies multi-head self-attention (6 heads, dimension 128) followed by a position-wise feed-forward network with hidden dimension 1,536 ($2 \times$ the feature dimension) and GELU activation. Dropout of 0.35 is applied throughout. Pre-norm formulation (layer normalization before attention) is used for training stability.

Formally, for each encoder layer:

$$X' = Z + \text{FFN}(\text{LN}(Z)), \quad (5)$$

$$Z = X' + \text{MHA}(\text{LN}(X')), \quad (6)$$

where MHA denotes multi-head attention and FFN the feed-forward network.

- Classification head: The output at the CLS token position is passed through a final layer normalization and a two-layer MLP:

$$\hat{y} = \text{Linear}_{256 \rightarrow 3} \circ \text{Dropout} \left(\text{GELU} \left(\text{Linear}_{768 \rightarrow 256} \left(\text{LN}(X'_{[:,0,:]}) \right) \right) \right) \quad (7)$$

The softmax of $\hat{y}$ gives the predicted class probabilities. The total number of trainable parameters is approximately 14 million.

At inference time, a newly detected bee will have fewer than $T = 16$ frames in its buffer. Rather than waiting for the buffer to fill before classifying, the sequence is padded to length T by repeating the last available feature vector. A boolean padding mask is passed to the Transformer's attention mechanism padded positions are masked out and do not contribute to self-attention computations,

preventing the model from treating repeated features as genuine temporal information. Classification begins once at least 4 frames have been accumulated ($\text{MIN_BUFFER} = 4$), balancing responsiveness against reliability.

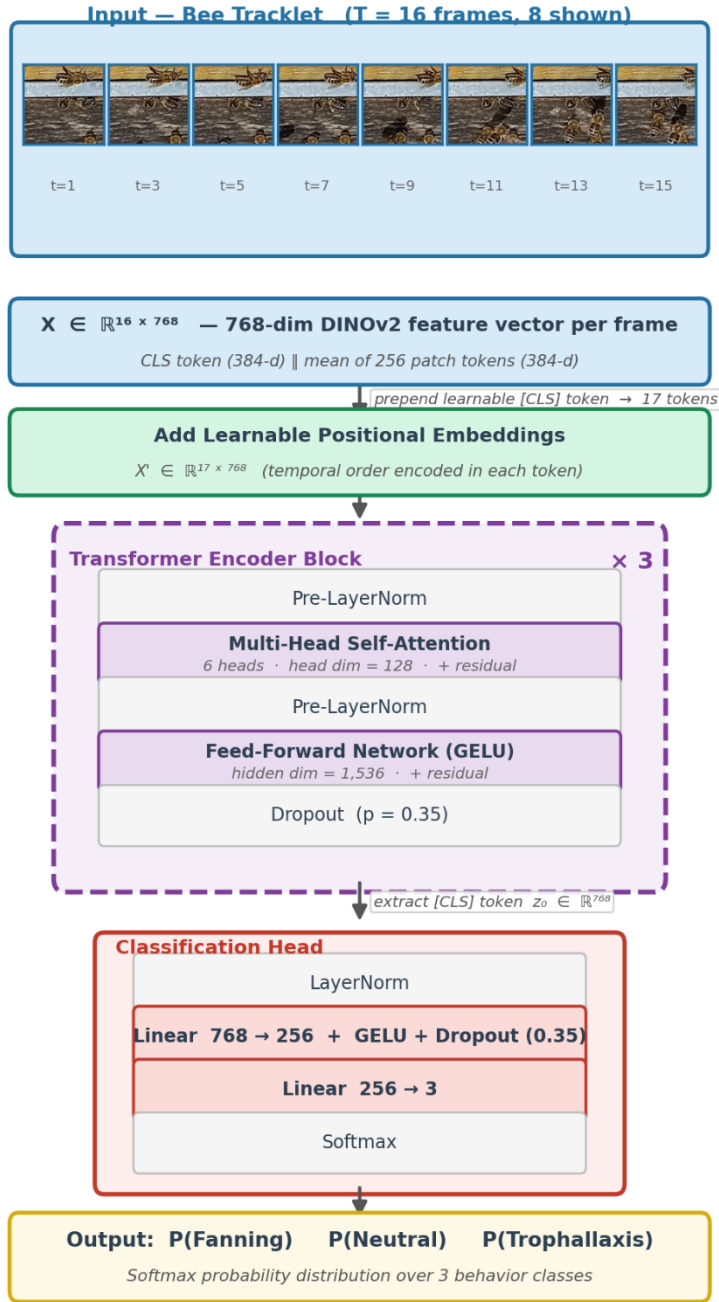


Fig 3.3. Temporal Sequence classifier architecture diagram

3.5.2 Training

The classifier was trained with the AdamW optimizer (learning rate 1×10^{-4} , weight decay 0.1) with cosine annealing over a maximum of 150 epochs, linear warmup over the first 10 epochs, and early stopping with patience of 25 epochs. Mixed-precision training (AMP) with gradient clipping at norm 1.0 was used throughout.

- Proportional multi-window sampling: A critical design decision governs how training samples are drawn from each sequence. Rather than drawing one random window per sequence per epoch which treats a 3,000-frame track identically to a 3-frame track each sequence contributes a number of training entries proportional to its length: $n_{\text{windows}} = \max(1, \lfloor L/T \rfloor)$, where L is the sequence length. At each training step, each entry independently samples a random starting position within its source sequence. This ensures that long, information-rich tracks are represented proportionally more often, converting what would otherwise be a handful of minority-class samples per epoch into thousands of diverse training windows from the same physical bee.
- Loss function: Cross-entropy with label smoothing $\epsilon = 0.1$ and class weights inversely proportional to the number of training samples per class (after proportional window expansion):

$$w_c = \frac{N_{\text{total}}}{C \cdot N_c}, \quad (8)$$

Where N_c is the number of training windows for class c after expansion and $C = 3$. With proportional multi-window sampling, the expanded sample counts are much more balanced than raw sequence counts, yielding moderate weights: $w_{\text{fan}} \approx 0.93$, $w_{\text{neu}} \approx 0.66$, $w_{\text{tro}} \approx 2.40$. Label smoothing prevents the model from assigning near-unity softmax probabilities to correctly predicted samples, improving calibration.

- MixUp regularization: On 50% of training batches, two randomly selected samples are linearly interpolated in feature space with mixing coefficient $\lambda \sim \text{Beta}(0.6, 0.6)$. The loss for a mixed sample is computed as the weighted sum of cross-entropy losses for the two constituent labels. MixUp forces the model to learn smooth decision boundaries and prevents overconfident predictions for memorised feature combinations.

Additional augmentations: Feature-space augmentations applied per sample

- Gaussian noise $\sigma = 0.05$
- Temporal reversal = 50% probability
- Temporal Jitter = Up to 3 frames replaced by a neighbour
- Feature dropout = Up to 3 randomly selected real frames zeroed, simulating missed detections

On non-MixUp batches, R-Drop regularization runs two forward passes through the model with different dropout masks and adds the symmetric KL divergence between the two output distributions to the loss $\alpha = 0.2$, ensuring consistent predictions under dropout uncertainty.

3.6 End-to-End Integration

At inference time, the five stages (detection $\rightarrow$ tiling $\rightarrow$ tracking $\rightarrow$ feature extraction $\rightarrow$ classification) are chained into a single processing loop per frame. The action label and confidence for each track are updated once per frame and persisted between frames.

Predictions are not temporally smoothed each frame produces an independent prediction from the current buffer state. No running average or majority-vote filter is applied across frames; the latest softmax output is displayed directly.

A confidence threshold of 0.70 is applied to the softmax output predictions below this threshold are labeled "uncertain" rather than assigned a behavior class. This threshold indicates that the model's top predicted class has a softmax probability below 0.70: the model is uncertain rather than confidently wrong. Uncertain predictions typically occur during early buffer fill-up (before sufficient frames have accumulated to form a stable feature representation) or at transitions between behaviors, when the rolling buffer contains a mix of frames from two different behavioral states.

If a track has not yet accumulated the minimum $\text{MIN_BUFFER} = 4$ frames required to trigger classification, no prediction is produced and no label is displayed for that track.

3.7 Chapter 3 Conclusion

The proposed pipeline separates bee detection and behavior classification into independently trainable stages connected by ByteTracker-assigned identities and a per-track feature buffer. The detection stage uses YOLOv11-medium with SAHI tiling to reliably locate small bees. Feature extraction is delegated to a frozen DINOv2-small backbone, with the trainable parameter count limited to approximately 14 million (the classifier only), preventing overfitting on a small behavioral dataset. The Temporal Sequence Classifier uses a pre-norm Transformer encoder to learn temporal patterns over 16-frame windows. The combination of class weighting, random window sampling, and feature-space augmentation addresses the extreme class imbalance in the training data.

4. DATA SCREENING

This chapter describes the source video material, the construction of both datasets used in this thesis, and the augmentation strategies applied to each. It is relevant to readers who need to understand the data collection criteria, the temporal structure of the behavioral sequences, and the design decisions made to address class imbalance and overfitting. The chapter concludes with a summary of the data preparation challenges and their mitigations.

4.1 Source Video Characteristics

All training and evaluation data were sourced from video recordings of a honeybee colony hive entrance captured under natural outdoor lighting conditions.

Frames were extracted from video at a sampling rate of one frame every six frames. At 50 fps, consecutive frames are nearly identical a bee walking at typical speed moves only 3–5 pixels between adjacent frames, meaning frame N and frame $N+1$ carry negligible new information. Sampling every sixth frame ensures each extracted image is visually distinct from the previous one: a walking bee has moved approximately 20–30 pixels and any wing motion has progressed to a different phase. This stride is consistent with standard practice in video understanding published action recognition benchmarks such as Kinetics and UCF101 apply comparable strides (4–16 frames) on 25–30 fps footage. A secondary benefit is dataset size reduction: stride-6 sampling reduces annotation effort and storage requirements by approximately $6\times$ relative to consecutive-frame extraction. All bees visible in each extracted frame were annotated regardless of their behavioral state.



Figure 4.1. Representative frame from source video.

4.2 Detection Dataset

The detection dataset was collected from video recordings of a hive entrance. Images were extracted from video at a sampling rate of one frame every six frames, consistent with the stride-6 strategy described in Section 4.1. A single object class bee was annotated using bounding boxes in YOLO format (normalized center x, center y, width, height). All bees visible in each frame were annotated regardless of their behavior.

Raw video frames were preprocessed with 'Utilities/tiling.py' to produce 640×640 px tiles the native input resolution of the YOLOv11 detector. Tiling was performed with 25% overlap to ensure that bees near tile boundaries appeared in at least two tiles, reducing missed detections at the edges. This step serves a dual purpose: it generates a larger number of training images from a limited number of source frames and presents bees at a larger apparent scale, improving the quality of gradient signal during training. The tiled dataset was stored in 'Data/DET_data_sliced/'.



Figure 4.2. Before/after tiling.

The tiled dataset was split into training, validation, and test subsets using 'Utilities/data_split.py'. The split was performed at the source-video level to prevent frames from the same recording session appearing in both subsets a critical precaution given the high visual similarity of adjacent frames from the same video:

Table 4.1: Tiled dataset statistics

Split	Images
Train	21,000
Validation	4,500
Test	4,501
Total	30,001

4.3 Action Recognition Dataset

4.3.1 Behavior Classes

Three behavior classes were defined:

- **Fanning:** A bee fans its wings rapidly while stationary, ventilating the hive or dispersing pheromones. Visually characterized by rapid wing motion and a stationary body.
- **Trophallaxis:** Two or more bees make sustained mouth-to-mouth contact to exchange food or liquid. Visually characterized by bees facing each other with antennae contact and minimal movement.
- **Neutral:** A bee moves normally at the entrance with no distinctive behavioral signature: walking, entering, or exiting the hive.

The fanning and trophallaxis classes are biologically meaningful signals of colony state. The neutral class provides a necessary baseline without it; the classifier would be trained only to distinguish between two rare behaviors and would have no concept of ordinary activity.

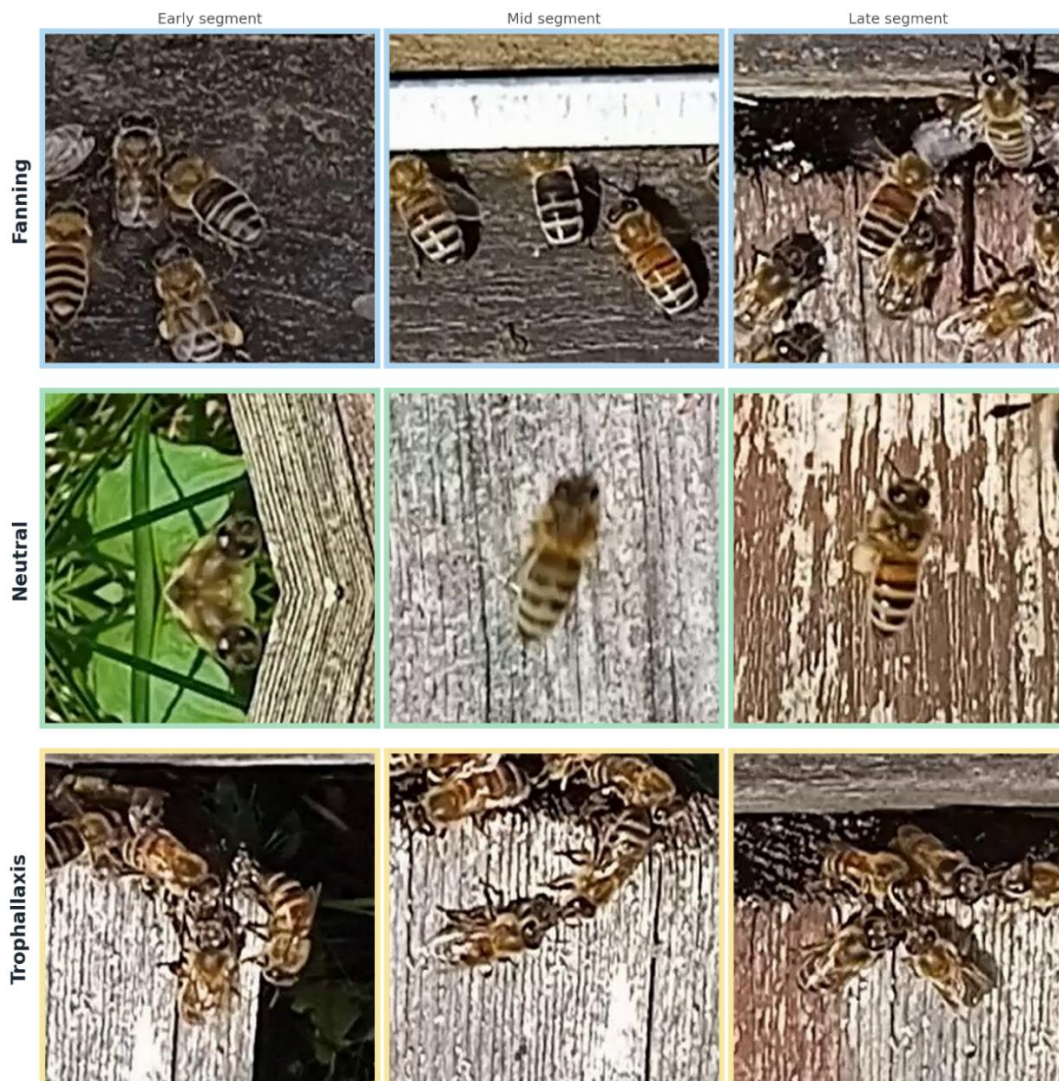


Figure 4.3. Sample crops per class.

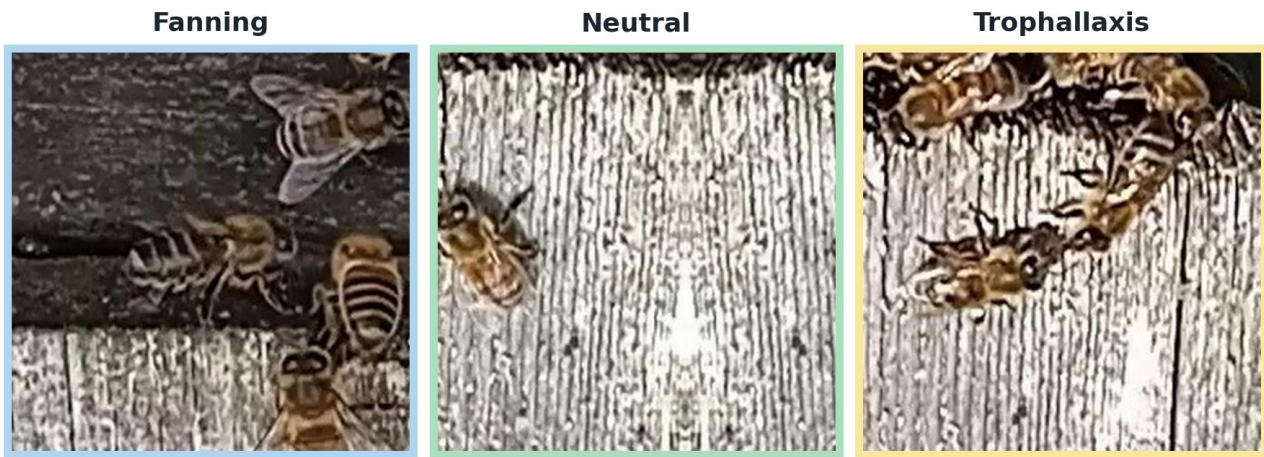


Figure 4.4. Additional dataset samples.

4.3.2 Dataset Construction

Fanning and trophallaxis behaviors were identified manually from hive entrance footage. Individual bees engaged in each behavior were annotated with YOLO-format bounding boxes. Crops were then extracted automatically by `data\_prep.py`, which processes the annotated frames in strict chronological order within each source video and assigns bee identities using ByteTracker the same tracker used at inference time. Crops are centered on the ByteTracker-assigned bounding box with reflect-padding applied when the bee was near the frame border. Each crop is saved with a structured filename encoding the source video, frame number, and ByteTracker track ID:

- Fanning and trophallaxis: `{class}\_{video\_id}\_{frame:05d}\_{track\_id}.jpg`
- Neutral: `neutral\_{video\_id}\_tile{tile\_id}\_{det\_id}\_{global\_frame}.jpg`

This naming convention allows the dataset to be reconstructed into temporally ordered sequences per bee without storing explicit sequence metadata the frame number acts as a sort key, and the (video_id, track_id) pair acts as a group key.

The use of ByteTracker for `track\_id` assignment is a deliberate design choice. An earlier version of the pipeline grouped crops by annotation index within the YOLO label file the order in which annotations appear in the `.txt` file rather than by a tracked identity. This produced spurious sequences in which annotation index 0 in frame t and annotation index 0 in frame $t + 1$ corresponded to different physical bees, since annotation order within a frame is arbitrary. ByteTracker resolves this by maintaining persistent IoU-based identity across consecutive frames: the same physical bee receives the same track ID across its entire visible trajectory, ensuring that the resulting sequence contains feature vectors from one individual rather than from a random mixture of individuals sharing the same annotation index.

The neutral class was not drawn from unlabeled footage. It was sampled from `DET\_data\_sliced\_split` using the existing YOLO bounding box annotations, targeting 25% of total crops ($\text{neutral_RATIO} = 0.25$) with random seed = 69 for reproducibility. Neutral crops therefore represent bees already localized by the detector but not annotated as fanning or trophallaxis ordinary walking, entering, or exiting bees captured in the same tiled-frame format as the behavioral classes. The action recognition validation split was inherited from the source AR_dataset, which already had train and val subfolders established during annotation preserving the source-video level split structure that prevents frames from the same recording session from appearing in both subsets, consistent with the detection dataset split strategy.

The relatively small number of trophallaxis training sequences (76) reflects the biological rarity of the behavior rather than a data collection shortcut. Trophallaxis is brief and infrequent at the hive entrance relative to normal foraging activity. The source AR_dataset contained 13,506 annotated trophallaxis images, but almost all came from a small number of distinct recording events with many frames per event, after grouping by (video_id, track_id) under ByteTracker, 76 unique sequences remain. Scarcity of events, not images, is the limiting factor. This contextualizes why $F1 = 0.96$ achieved with 76 sequences is a meaningful result: the proportional multi-window sampling strategy and class weighting compensate for the gradient imbalance, and DINOv2 features are rich enough that the model needs only a limited number of examples to identify trophallaxis' distinctive visual signature.

The resulting image-level class distribution is as follows:

Table 4.2. Action dataset statistics

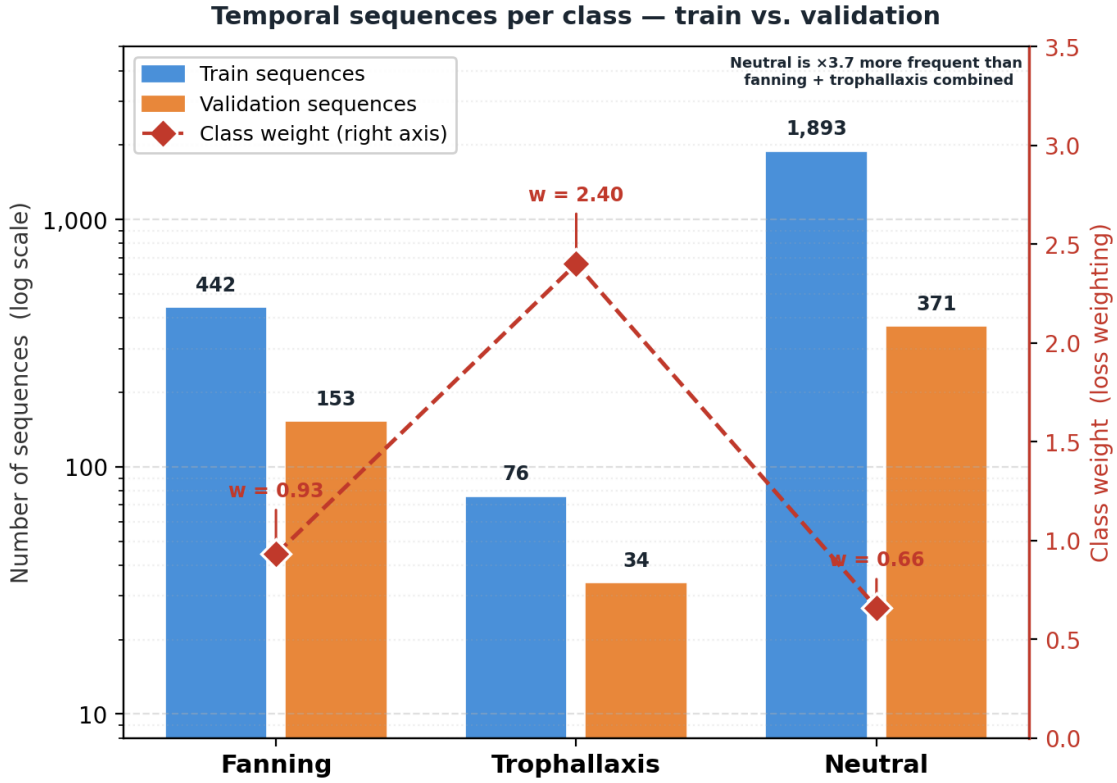
Class	Train	Validation	Total
Fanning	47,106	9,596	56,702
Trophallaxis	18,344	3,369	21,713
Neutral	22,065	4,408	26,473
Total	87,515	17,373	104,888

4.3.3 Dataset Statistics

The AR_v2_dataset images group into 2,969 temporal sequences (2,411 train / 558 validation) when organised by source video and ByteTracker track ID. The meaningful unit for behavior classification is not an individual image but a temporally ordered sequence from a single bee; sequences were constructed by grouping filenames by (video_id, track_id) and sorting by frame number. Sequence lengths range from 4 to 2,227 frames (training median: 7; validation median: 8), reflecting the variable duration of behavioral events captured by ByteTracker:

Table 4.3. Action sequences statistics

Class	Train Sequences	Validation Sequences
Fanning	442	153
Trophallaxis	76	34
Neutral	1,893	371
Total	2,411	558

**Figure 4.3.3** Grouped bar chart of sequence counts per class with class weights overlay.

The imbalance at the sequence level neutral comprising approximately 79% of training sequences reflects the natural distribution of behaviors at the hive entrance: fanning and trophallaxis are genuinely rare events. Compared to the earlier dataset version (where neutral exceeded 98% of sequences), ByteTracker's shorter track segmentation and a wider behavioral annotation effort have produced a more balanced distribution, though a meaningful imbalance remains. This imbalance is addressed primarily through proportional multi-window sampling (Section 3.5.2): each sequence contributes training windows proportional to its length, so long behavioral tracks are represented far more often per epoch than their raw sequence count would suggest. Supplementary class weighting provides a residual correction with moderate weights: $w_{fan} \approx 0.93$, $w_{tro} \approx 2.4$, $w_{fan} \approx 0.66$. These weights are moderate rather than extreme because proportional multi-window sampling has already substantially rebalanced the effective training distribution. Oversampling was not used as an alternative because repeating sequence windows would distort the temporal structure of training data,

duplicate or near-duplicate windows would introduce artificial patterns that the model could exploit rather than learning genuine behavioral features.

DINOv2 feature extraction was run once over all images in the dataset, producing a 768-dimensional feature vector (CLS, mean-patch) per image. Features were saved to `features\_v3.pkl` (approximately 320 MB). Pre-computing features rather than running DINOv2 at training time freezes the feature space so that augmentation operates on a clean low-dimensional representation rather than raw pixel data, and reduces each training epoch to a series of linear algebra operations on cached vectors making training fast despite the large image count.

4.4 Data Augmentation

4.4.1 Detection Augmentation

The detection model was trained with a medium-intensity augmentation preset combining geometric and photometric transforms:

Table 4.4. Detection augmentation

Augmentation	Value	Rationale
Rotation	$\pm 10^\circ$	Camera mounting angle variation
Translation	15%	Variable bee position within tile
Scale	$\pm 30\%$	Distance variation between hive and camera
Horizontal flip	50%	Symmetric – no preferred hive orientation
HSV saturation	$\pm 50\%$	Cloudy versus sunny conditions
HSV brightness	$\pm 30\%$	Dawn to dusk lighting variation
Mosaic	50%	Improves small-object detection
Copy-paste	35%	Synthetically increases bee density

Mosaic augmentation: Combines four training images into a single composite by resizing each to a quarter of the target resolution and arranging them in a 2×2 grid. For small-object detection this is valuable for two reasons: it increases the number of annotated objects per training sample, giving the model denser gradient signal per forward pass; and it presents each object across a wider range of scales, positional contexts, and adjacent backgrounds within a single image. Even at a moderate

application probability, mosaic substantially diversifies the training distribution because it alters the fundamental structure of training samples four independent images become one rather than applying minor perturbations to a single source image.

Copy-paste augmentation: extracts annotated bees from one image and composites them into another at random positions. In this dataset the number of bees per frame is limited and their spatial distribution is biased toward the hive entrance area. Copy-paste synthetically increases bee instance count per image and introduces bees into spatial contexts tile positions and background textures that may not appear naturally in the source footage. This is particularly effective at reducing overfitting to background correlations (e.g., the hive landing board texture that co-occurs with bees in nearly every training image). Augmenting at 35% probability ensures that a substantial fraction of training images contain synthetically placed bees in atypical spatial contexts, improving generalization to the full frame area.

Vertical flipping was disabled the hive orientation is fixed and the detector should not learn inverted-gravity features.

4.4.2 Action Recognition Augmentation

Because the AR classifier trains pre-computed feature vectors rather than raw images, augmentations are applied in feature space:

- Gaussian noise ($\sigma = 0.05$): Adds perturbations to each feature vector, simulating visual variation caused by lighting changes, motion blur, or subtle bee appearance differences across frames.
- Temporal reversal (50% probability): Reverses the frame order within a window. Fanning wing motion is cyclical and direction-invariant; trophallaxis is approximately symmetric. This prevents the model from relying on the direction of change rather than its presence.
- Temporal Jitter: (up to 3 frames replaced by a neighbour) Simulates duplicate frames or missed detections during real inference
- Feature Dropout: (up to 3 random real frames zeroed) Simulates complete detection failures, forcing the model to make reliable predictions from partial temporal evidence.
- MixUp: (50% of batches, $\lambda \sim \text{Beta}(0.6, 0.6)$) Linearly interpolates two feature sequences and their labels in feature space, regularising the decision boundary and preventing overconfident predictions.
- Label Smoothing: $\epsilon = 0.1$ replaces hard one-hot targets with smooth distributions, directly preventing the model from assigning near-unity softmax confidence to training samples.
- R-Drop: ($\alpha = 0.2$, Non-MixUp batches) Penalizes the KL divergence between two forward passes under different dropout masks, ensuring consistent uncertainty estimates.

4.6 Chapter 4 Conclusion

Source video was recorded at the hive entrance under natural lighting conditions. Two purpose-built datasets were constructed: the detection dataset of 30,001 tiled images provides sufficient variety for reliable bee localization, and the action recognition dataset of 104,888 images organized into 2,969 temporally ordered sequences captures three behavior classes. The natural but severe class imbalance toward neutral is addressed through proportional multi-window sampling and supplementary class weighting ($w_{fan} \approx 0.93, w_{tro} \approx 2.4, w_{fan} \approx 0.66$) rather than resampling. Pre-computing DINOv2 features make classifier training fast and prevents backbone overfitting. Augmentation strategies were designed to reflect the specific visual and temporal variability expected at inference time.

5. INVESTIGATION OF DEEP LEARNING METHOD FOR BEE BEHAVIOR RECOGNITION

This chapter presents the results of all experiments conducted in this thesis. It is structured to first evaluate each stage of the pipeline independently detection, then action recognition before assessing end-to-end behavior. All results are compared against the prior VGTU work where applicable. This chapter is relevant to readers who need to assess the accuracy, reliability, and limitations of the proposed system.

5.1 Experimental Setup

All training was performed on a single consumer-grade NVIDIA GeForce RTX 4050 GPU using CUDA. Due to a per-epoch training time of 10–13 minutes for the detection model, a custom chunked training strategy was implemented: training was divided into fixed-length sessions, with state persisted between sessions via `'chunk_state.yaml'` (current epoch, best metric) and `'early_stop_state.yaml'` (patience counter). This allowed training to resume correctly without resetting YOLO's internal counters or losing the running metrics CSV which cannot be recovered if YOLO's native `'resume=True'` mechanism is used across session boundaries, as it clears all previously accumulated results. The action recognition classifier was trained in under two minutes per epoch and required no such workaround.

The detection model was trained using the Ultralytics framework; the action recognition model was implemented in PyTorch 2.x with the HuggingFace Transformers library for DINOv2 access. Evaluation metrics are:

Detection: precision, recall, mAP@0.5, and mAP@0.5:0.95 on the validation split.

Action recognition: Per-class precision, recall, F1-score, and macro-average F1 on the validation split.

5.2 Detection Results

5.2.1 Model selection

Before committing to the medium-scale architecture, an exploratory phase compared two model sizes (nano and small) under two optimizers (SGD and AdamW) using a lighter augmentation preset. Each model was trained for up to 200 epochs with early stopping patience of 20, reaching convergence between epochs 130 and 150.

Table 5.1. Model and optimizer comparisons

Model	Optimizer	mAP@0.5 (approx.)	mAP@.5:0.95 (approx.)
YOLOv11-nano	SGD	0.73	0.63
YOLOv11-nano	AdamW	0.78	0.68
YOLOv11-small	SGD	0.82	0.77
YOLOv11-small	AdamW	0.88	0.82

Across both architectures, AdamW consistently outperformed SGD by approximately 5 percentage points on both metrics. This consistency suggests that AdamW's decoupled weight decay provides more stable convergence for this dataset which contains noise from motion blur and varying lighting by preventing weight decay from interfering with gradient updates in the way that L2-regularized SGD can. AdamW was therefore selected as the optimizer for all subsequent training.

The performance gap between nano and small (~7–8 percentage points) indicated that model capacity was a binding constraint, motivating the upgrade to YOLOv11-medium for the final training run. Larger variants (large, xlarge) were not evaluated at 10–13 minutes per epoch for the medium model, training a larger variant would have required prohibitively long sessions given the chunked training setup. Values in the table above are reported as approximate because these were exploratory runs with a lighter augmentation preset and early-stopping patience of 20; exact reproduction was not the goal, only relative comparison between architectures and optimizers.

5.2.2 Final Model Performance

YOLOv11-medium was trained for 238 epochs before early stopping was triggered (patience = 50). The best checkpoint was selected by validation mAP@0.5:0.95. Results on the validation set are as follows:

Table 5.2. Final model comparisons

Metric	YOLOv11-medium (this work)	YOLOv8-medium (Sledevič et al. [3])
Precision	0.983	0.98
Recall	0.973	0.97
mAP@0.5	0.992	0.97
mAP@0.5:0.95	0.853	0.65

YOLOv11-medium achieves a substantial improvement in $\text{mAP}@0.5:0.95$ from 0.65 to 0.853 representing a 31% relative improvement in localization quality across IoU thresholds. This reflects both the improved backbone of YOLOv11 and the SAHI tiling strategy, which ensures that small bees are detected at their native scale.



Figure 5.1. Sample inference frame.

The improvement in $\text{mAP}@0.5:0.95$ over $\text{mAP}@0.5$ is notable: while both models achieve near-perfect detection at a 0.5 IoU threshold, YOLOv11 produces significantly tighter bounding boxes (higher IoU with ground truth), which is important for the downstream cropping step tighter crops contain less background noise for the feature extractor.

5.3 Action Recognition Results

5.3.1 Model Performance

The Temporal Sequence Classifier with a 16-frame buffer reached its best validation checkpoint at epoch 27 under the full regularization regime (label smoothing, MixUp, R-Drop, feature dropout):

Table 5.3. Buffer size comparisons

Buffer	Best Epoch	Val F1 (macro)	Accuracy	Val Samples
16 frames	27	0.964	0.97	1,829

Training was conducted with AdamW ($\text{lr} = 1 \times 10^{-4}$, weight decay = 0.1), linear warmup over 10 epochs, cosine annealing over a maximum of 150 epochs, and early stopping with patience = 25. The

regularization regime slows convergence and prevents the model from over-committing to high-confidence predictions on training data, producing well-calibrated outputs. The validation set covers the full sequence space at BUFFER_SIZE = 16 with stride 8.

5.3.2 Per-Class Results

Table 5.4. Best model statistics

Class	Precision	Recall	F1-score
Fanning	1	0.95	0.97
Neutral	0.95	1	0.98
Trophallaxis	0.91	0.99	0.95
Macro Avg	0.95	0.98	0.96

All three classes achieve F1-scores above 0.95. Fanning achieves perfect precision (1.00) with 0.95 recall when the model predicts fanning, it is never wrong, though it occasionally mis-classifies a fanning bee as another class. Neutral achieves perfect recall (1.00): no neutral bee is misclassified as a behavioral event, though a small fraction of behavioral detections are predicted as neutral (precision 0.95). Trophallaxis achieves 0.91 precision and 0.99 recall, the model reliably detects every trophallaxis event, with a modest false-positive rate where some non-trophallaxis bees are labelled as trophallaxis.

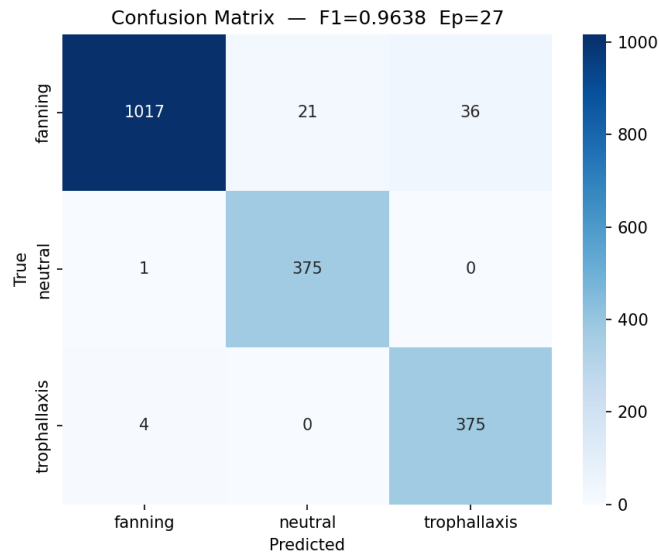


Figure 5.2. Confusion matrix

5.3.3 Discussion

The model reaches its best checkpoint at epoch 27 under a full regularization regime label smoothing, MixUp, R-Drop, and feature dropout all apply regularization pressure that slows convergence and prevents the model from over-committing to high-confidence predictions on training data. The regularization techniques directly address the overconfidence failure mode observed in earlier model

versions, where the model assigned near-unity softmax scores to incorrect predictions at inference time.

After epoch 27, validation F1 declined steadily falling to approximately 0.90 by the time early stopping triggered at epoch 52. This late-training drift is consistent with the high per-epoch variance characteristic of small-N training: each epoch samples a single random window per sequence, so the effective per-epoch training set for the minority classes is on the order of a few hundred windows, and minor variance in window selection accumulated over many epochs past the optimum produces the observed gradual degradation. The early-stopping mechanism correctly restored the epoch-27 checkpoint for all reported metrics.

The expanded feature representation (768-dim CLS || mean-patch) provides richer spatial information than the CLS token alone. For trophallaxis, where the diagnostic signal is the spatial arrangement of two bees in close contact rather than a global appearance change the mean patch token contributes local texture and relative position information that the global CLS token cannot encode. This is reflected in the high trophallaxis recall of 0.99: the model reliably detects every trophallaxis event, with the patch-level features consistently encoding the characteristic close-contact pose signature.

The fanning class achieves perfect precision (1.00): when the model predicts fanning, it is never wrong. Fanning's visual signature rapid wing motion producing characteristic motion blur in a stationary bee is distinctive in DINOv2's feature space and does not overlap with other classes. The small recall deficit (0.95) reflects a subset of fanning frames where wing motion is not yet visible (beginning of a fanning) or where the bee has partially moved out of the crop; these frames are classified as neutral, which is a correct-direction prediction.

Table 5.5. Earlier training iterations

Model	Dataset	Architecture	Class Weights	Best ep	Val F1
v1(buf_16)	AR_dataset_v1	384-d / 2 layers / 4 heads	Extreme (21.99, 68.06, 0.34)	9	0.9818
v2(extreme_16)	AR_dataset_v1	384-d / 2 layers / 4 heads	Extreme (21.99, 68.06, 0.34)	6	0.3878
v3(Final)	AR_dataset_v2	768-d / 3 layers / 6 heads	Moderate (0.93, 0.66, 2.40)	27	0.9638

Original dataset (AR_dataset_v1) where 'bee_id' encoded annotation index rather than a persistent tracked identity (see Section 4.3.2.). Sequences fed to v1/v2 therefore contained no genuine temporal coherence. Apparent high F1 reflects per-frame appearance classification on incoherent sequences rather than temporal behavior recognition on properly tracked individuals.

The iterative trajectory leading to v3 illustrates why this specific configuration was retained as final. The v1 model achieved an apparently high validation F1 of 0.9818, but on the original AR_dataset where 'bee_id' encoded a line number within each YOLO label file rather than a persistent tracked identity (Section 4.3.2). The "temporal sequences" fed to v1 therefore contained no genuine temporal coherence: the model learned to classify behavior from per-frame appearance alone, which is the only task feasible on that data, and the metric reflects that simpler task rather than the temporal recognition the architecture nominally addresses. A subsequent v2 iteration, trained on the same broken dataset with adjustments to the regularization scheme, and intense overfitting measures similar to v3 final model, failed to converge (validation F1 = 0.3878), demonstrating that intermediate fixes were insufficient. Only the v3 configuration combining the corrected AR_v2_dataset, proportional multi-window sampling in place of extreme class weighting, an expanded 768-dimensional feature representation, increased model capacity, and the regularization stack described above simultaneously trained successfully and avoided the overconfidence pathology observed in v1.

The proportional multi-window sampling strategy is the primary driver of the performance improvement. By ensuring that long fanning and trophallaxis sequences contribute proportionally more training windows, the effective number of behavioral training examples per epoch increases from approximately 86 (one per sequence) to thousands of diverse windows from genuine behavioral tracks. The moderate class weights under this regime (approximately 0.93, 0.66, 2.40) produce a stable gradient signal without the extreme weight-induced variance seen in v1.

5.4 End-to-End Pipeline Evaluation

The complete pipeline SAHI detection, ByteTracker, DINOv2 feature extraction, and Temporal Sequence Classifier were evaluated on the full fanning (18,000 frames) and trophallaxis (20,480 frames) source datasets from 'AR_dataset', providing a large-scale cross-behavior assessment of integrated pipeline performance. The evaluation introduces two methodological advances over a naive per-detection accuracy measure.

One methodological caveat applies to all results that follow: the evaluation was run on every sixth frame of the source videos rather than on true consecutive frames, to keep the computational cost of SAHI tiling, DINOv2 feature extraction, and Transformer classification tractable across the 44,680-frame evaluation set. ByteTracker's 30-frame lost-track buffer therefore sees a denser detection stream per unit of wall-clock time than it would under true 30-fps inference, modestly favoring tracking continuity; the 16-frame Transformer buffer also spans a wider real-time interval per fill. The implications for true continuous-video deployment are discussed further in Section 6.3.

5.4.1 Detection Stage Reliability

YOLO detection confidence remained consistently high across both evaluation datasets, with mean confidence 0.893 for the fanning dataset and 0.887 for the trophallaxis dataset both well above the 0.45 detection threshold. Confidence distributions were tightly concentrated above 0.85, confirming that the detection stage operates reliably under both behavioral conditions.

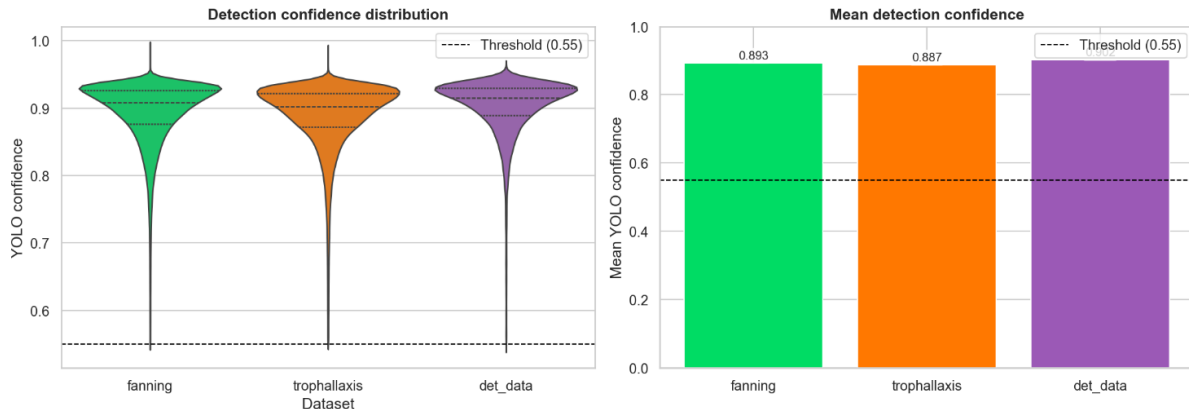


Figure 5.3. Detection confidence distributions

5.4.2 Pipeline-level Classification Accuracy

Evaluation methodology: A direct comparison of per-detection predictions against scene-level video labels is methodologically unsound: a video labeled "fanning" contains bees in many behavioral states simultaneously walking, entering, exiting, and occasionally performing other behaviors. A detector identifying 27 bees per frame across an 18,000-frame fanning video will include many bees that are objectively not fanning at that moment; labeling any non-fanning prediction for these bees as an error conflates scene-level annotation with per-bee ground truth.

To address this, the evaluation uses IoU-based matching against the per-frame YOLO annotations present in `AR\_dataset`: each pipeline detection is considered a valid evaluation sample only if its bounding box overlaps a ground-truth annotated bee at $\text{IoU} \geq 0.15$. Detections not matched to any GT box background bees not annotated as performing the target behavior are excluded from accuracy computation. This IoU-matched metric measures the classifier's accuracy on the specific bees that were confirmed by a human annotator as exhibiting the target behavior.

Scene-level majority vote: In parallel, scene-level classification accuracy is assessed by taking the modal predicted class across all classified (non-buffering) detections in a dataset and comparing it to the video's ground-truth behavior label. This metric answers the operationally relevant question: given the entire prediction distribution across a behavioral video, does the pipeline correctly identify the dominant behavior.

The IoU-matched accuracy column below counts uncertain predictions as errors any IoU-matched detection that did not receive a confident behavior label is treated as an incorrect prediction and therefore represents a conservative lower bound on classifier performance. Figure 5.4.4 reports the

less conservative figure using the same IoU-matched population but excluding uncertain predictions from the denominator.

Table 5.6. Pipeline statistics

Dataset	GT-matched detections	IoU-matched accuracy	Scene-level majority vote
Fanning video	58,601 / 271,828 (21.6%)	67.6%	Fanning
Trophallaxis Video	35,307 / 536,531 (6.6%)	88.0%	Trophallaxis

The scene-level majority vote is correct for both datasets: the modal predicted class in the fanning dataset is fanning, and the modal predicted class in the trophallaxis dataset is trophallaxis. This confirms that the pipeline correctly identifies the dominant behavioral signal present in each scene.

Figure 5.4. presents the broadest possible view, the prediction distribution across all non-buffering detections in each scene, including background bees not annotated as performing the target behavior. No IoU filtering is applied, every detected bee contributes to the distribution regardless of whether it was confirmed as exhibiting the target behavior by a human annotator. In the fanning dataset, the modal predicted class is fanning (32.0%), with neutral (28.4%), trophallaxis (26.9%), and uncertain (12.7%) making up the remainder. In the trophallaxis dataset, trophallaxis is the clear modal class at 51.6%, with fanning (18.3%), neutral (20.2%), and uncertain (9.9%) distributed across the rest. The spread across classes particularly the near-equal distribution in the fanning scene is expected rather than a failure: a typical hive-entrance frame contains 20–30 bees simultaneously, the vast majority of which are walking, entering, or exiting the hive and will correctly be predicted as neutral or uncertain. The target behavior (fanning or trophallaxis) is being performed by only a small subset of the visible bees at any moment, and those bees appear in every frame. The key result is that the target class is the modal prediction in both scenes despite this dilution, confirming that the pipeline produces a detectable behavioral signal even when evaluated over the full unlabeled population.

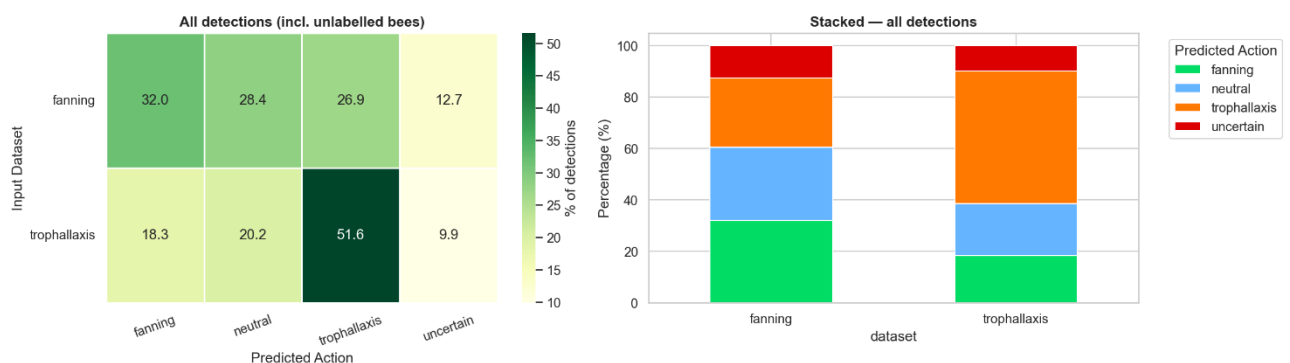


Figure 5.4 All-detections prediction distribution

Figure 5.5. narrows to the methodologically sound per-bee metric, only detections overlapping a human-annotated ground-truth bee ($\text{IoU} \geq 0.15$) are scored, removing all background bees from the evaluation. This IoU-matched population is the basis for all accuracy figures that follow. The effect of the filtering is substantial. In the fanning dataset, the correctly labelled fraction rises from 32.0% to 67.6%; in the trophallaxis dataset it rises from 51.6% to 88.0%. The improvement confirms that the background bees in Figure 5.4.2 were indeed the source of dilution: the pipeline was correctly classifying them as neutral or uncertain, not misclassifying them. When attention is restricted to bees that a human annotator confirmed as performing the target behavior, the classifier's accuracy is considerably higher. The error pattern also differs between classes, fanning errors are distributed across neutral (14.4%) and uncertain (10.1%), reflecting frames at the start or end of a fanning bout where wing motion is absent or ambiguous; trophallaxis errors are smaller in magnitude (6.2% neutral, 4.9% fanning), consistent with the distinctive visual signature of two bees in sustained mouth contact being reliably separable from the other classes.

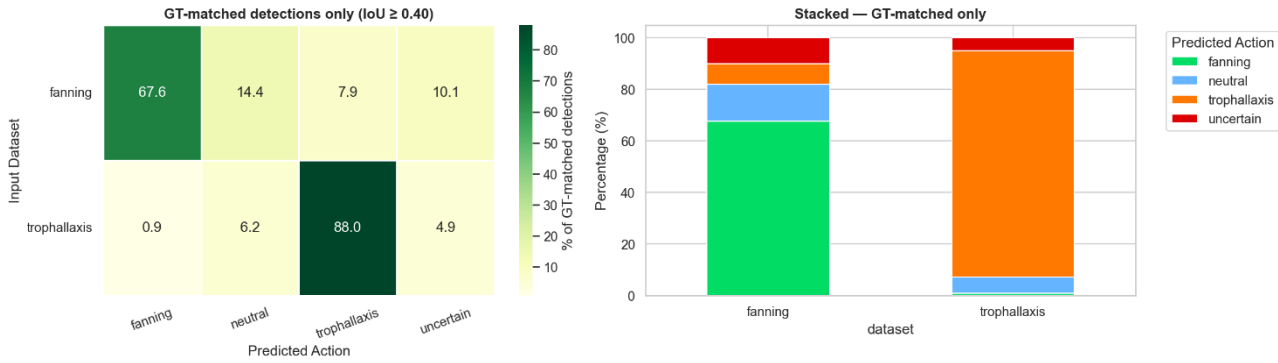


Figure 5.5. IoU-matched prediction distribution

Figure 5.6. restricts the evaluation to IoU-matched detections that received confident prediction uncertain outputs are excluded from the denominator and reports the primary end-to-end accuracy figures for the pipeline. The right panel shows these headline results: 75.2% of fanning bees and 92.6% of trophallaxis bees are correctly classified ($n = 52,695$ and $n = 33,561$ respectively). The substantially higher accuracy on trophallaxis reflects the visual distinctiveness of sustained two-bee contact, which the DINOv2 patch features consistently encode whereas fanning, whose signature is transient wing-blur, produces more frames where the classifier correctly defers to neutral. The left panel examines the softmax confidence distributions behind these results: for fanning, the model is more confident when it is right (mean 0.855) than when it is wrong (mean 0.746); the same pattern holds for trophallaxis (0.908 correct vs. 0.703 off-target). Both figures are well above the 0.55 classification threshold, indicating that most errors are not low-confidence guesses but confidently wrong predictions the model commits to an incorrect class rather than abstaining. This is consistent with the error patterns seen in Figure 5.4.3: fanning errors are predominantly neutral or uncertain predictions during behaviorally ambiguous frames, not high-confidence cross-class confusions.

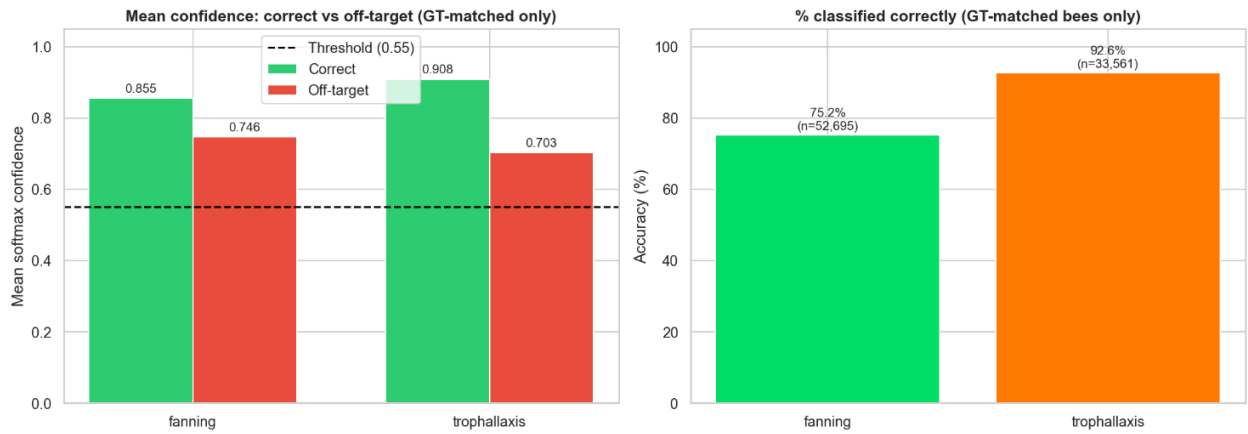


Figure 5.6. Action classification confidence on IoU-matched bees

5.4.3 Track Lifetime Analysis

Track lifetime distributions reveal that a substantial fraction of tracked bees produce trajectories too short to trigger classification. Figure 5.4.5 breaks tracks into three categories: fewer than 4 frames (never classified, as the minimum buffer threshold is not reached), 4–15 frames (partial buffer classification begins but without a full 16-frame window), and 16 or more frames (full buffer). In the fanning dataset, 32.4% of tracks never reach classification, 34.2% classify from a partial buffer, and only 33.5% accumulate a full buffer. Trophallaxis shows a more favourable distribution: 26.0% never classified, 23.9% partial, and 50.1% full reflecting the sustained nature of trophallaxis events, where two bees remain in contact for extended durations and therefore stay visible and trackable across more frames. The detection dataset shows the worst completion: 39.0% of tracks never classify and only 15.1% reach the full buffer, consistent with faster-moving foragers that pass through the frame quickly. The right panel confirms that 67.6% of fanning tracks and 74.0% of trophallaxis tracks reach the minimum threshold of 4 frames and are classified at least once. The fraction of tracks that never receive a prediction roughly one-third for fanning, one-quarter for trophallaxis — is the primary source of the gap between isolated classifier performance (macro F1 = 0.96) and pipeline-level accuracy: these bees are not misclassified, they simply produce no classification output at all due to tracker fragmentation.

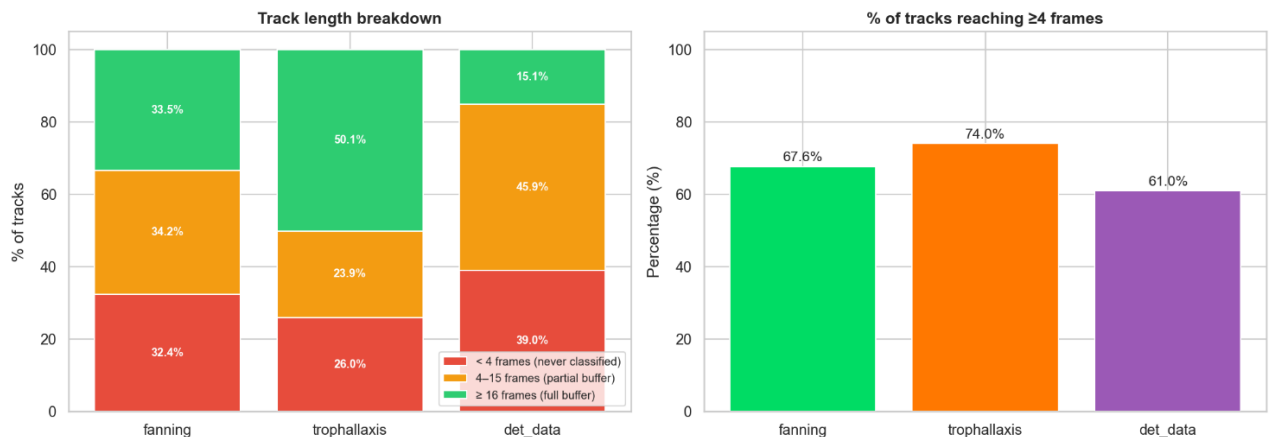


Figure 5.7. Track lifetime distributions

Figure 5.8. extends the analysis to the track level: for each track, the modal predicted class over all its classified frames is taken as the track's behavior label the track-level majority vote. This is more informative than per-detection accuracy because it integrates temporal evidence across the full visible trajectory of a single bee rather than treating each frame independently.

The left panel shows the distribution of per-track majority-vote labels across each scene. In the fanning dataset, 36.6% of tracks are majority-voted as fanning, the largest single class and the correct modal label with neutral (32.5%) and trophallaxis (30.8%) splitting the remainder. In the trophallaxis dataset, the signal is stronger: 57.3% of tracks are majority-voted as trophallaxis, again the dominant and correct class. The right panel reports per-track accuracy strictly: 21.3% of fanning tracks (1,110 / 5,203) and 43.6% of trophallaxis tracks (2,712 / 6,226) have a majority vote that matches the scene label. These figures are low by design most tracked bees in any scene are neutral walkers whose majority vote correctly resolves to neutral, not to the target behavior. A low per-track accuracy against the scene label does not indicate widespread misclassification; it reflects the natural composition of the hive entrance where behavioral events are rare. The operationally meaningful result is that the scene-level modal label is correct in both cases, and that trophallaxis being a sustained, visually distinctive event produces a more concentrated per-track signal (43.6%) than fanning (21.3%), consistent with the track lifetime data in Figure 5.4.5 showing that trophallaxis tracks are longer and accumulate more evidence per trajectory.

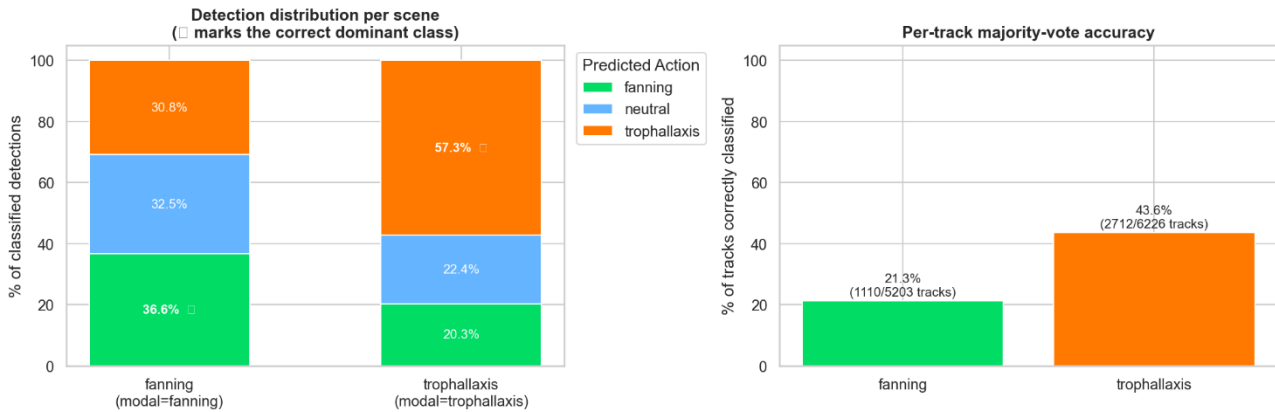


Figure 5.8. Track lifetime distributions

Figure 5.9. evaluates temporal post-hoc smoothing on the same population as Figure 5.4.4 IoU-matched bees with confident predictions so the no-smoothing baseline reproduces those figures directly (75.2% fanning, 92.6% trophallaxis). The question is whether averaging predicted probability distributions over a sliding window of $w = 3, 5$, or 7 consecutive frames improves these headline accuracy figures. The result is negative: smoothing produces negligible improvement across all window sizes. On the fanning dataset, accuracy increases from 75.2% (no smoothing) to 75.5% at $w = 7$ a difference of 0.3 percentage points. On the trophallaxis dataset, the improvement is similarly marginal: 92.6% to 92.8%. These gains are not practically meaningful.

This finding is informative rather than merely negative. The absence of benefit from post-hoc smoothing confirms that the temporal integration needed for reliable classification is already performed inside the model: the 16-frame rolling buffer feeds a Transformer encoder whose self-attention mechanism aggregates information across all buffered frames simultaneously. Smoothing consecutive softmax outputs adds a second layer of temporal averaging over already-temporally-integrated predictions a redundant operation. In contrast, action recognition systems that classify individual frames independently rely on post-hoc smoothing to achieve temporal consistency; the buffer-based design of this pipeline eliminates that dependency.

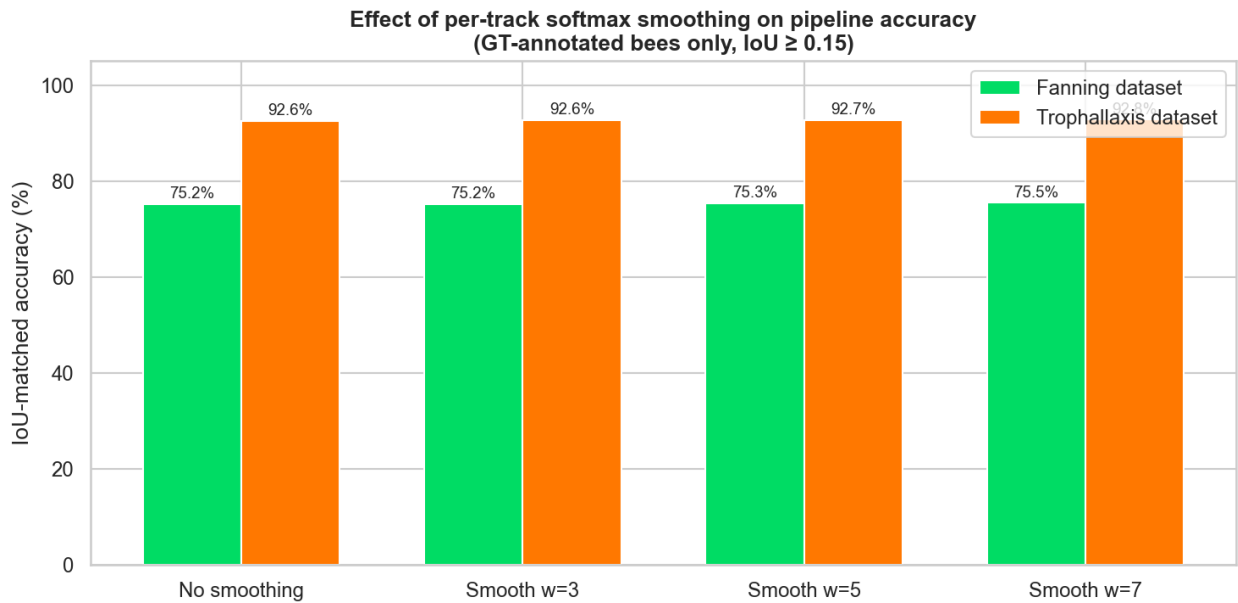


Figure 5.9. Temporal smoothing comparison

Taken together, the end-to-end evaluation presents a consistent picture. The detection stage operates reliably across all conditions, with mean YOLO confidence above 0.88 in both behavioral datasets. The pipeline's action recognition accuracy, measured on confirmed target bees (IoU ≥ 0.15) where the classifier produced a confident prediction, is 75.2% for fanning and 92.6% for trophallaxis these are the primary end-to-end classification figures. When uncertain predictions are retained in the denominator and counted as errors rather than abstentions, the figures reduce to 67.6% and 88.0% respectively, forming a conservative lower bound; the difference reflects approximately 10% of IoU-matched detections where the classifier correctly deferred to uncertainty rather than committing to a potentially wrong label. The gap between fanning and trophallaxis accuracy reflects the transient, frame-level ambiguity of wing-blur versus the sustained, spatially distinctive signature of bee-to-bee contact. Scene-level majority vote is correct for both datasets without exception: the pipeline correctly identifies which behavior dominates a given recording. Post-hoc temporal smoothing produces no meaningful improvement accuracy increases from 75.2% to 75.5% at $w = 7$ for fanning and from 92.6% to 92.8% for trophallaxis confirming that the 16-frame Transformer buffer already captures all the temporal context the classifier needs. The primary factor limiting pipeline-level performance

below the isolated classifier benchmark (macro F1 = 0.96) is tracker fragmentation: roughly one-third of fanning tracks and one-quarter of trophallaxis tracks never accumulate enough frames to trigger a classification output, contributing silence rather than errors. These bees are not misclassified they simply never enter the classifier. Addressing tracker robustness in high-density and close-contact scenarios is therefore the clearest path to closing the gap between isolated and pipeline-level performance.

5.5 Comparison to Prior Work

The most significant advance over prior work is not just the detection improvement while the 31% relative gain in mAP@0.5:0.95 is substantial, precision and recall gains are marginal but the replacement of heuristic inference with a trained classifier. Prior systems could not produce a behavior F1 because there was no ground-truth classifier to evaluate against; trajectory statistics and density maps do not yield a confusion matrix. The trophallaxis column in the table represents an entirely new capability, not an incremental improvement on an existing one: individual-level trophallaxis recognition from hive entrance video had not been demonstrated in the bee monitoring literature prior to this work.

Table 5.7. Work comparison

Capability	Sledevič et al. [3, 4, 16]	Vdoviak & Sledevič [18]	This work
-----	-----	-----	-----
Detection Architecture	YOLOv8-medium	YOLOv8-medium (temporal)	YOLOv11-medium
mAP@0.5 (detection)	0.97	0.955(trophallaxis)	0.992
mAP@0.5:0.95 (detection)	0.65	-	0.853
Behavior inference	Heuristic (speed, heat maps, density)	Detection-based per region	Learned per-bee classifier
Behavior classes	Foraging, fanning, guarding, washboarding	Trophallaxis	Fanning, trophallaxis, neutral
Individual-level attribution	Partial (track-based heuristic)	No (region-based)	Yes (per tracked bee)
Trophallaxis recognition	Not addressed	F1 not reported	F1 = 0.96
Behavior F1 (macro)	Not reported	Not reported	0.96

5.6 Chapter 5 Conclusion

YOLOv11-medium achieves strong detection performance with $\text{mAP}@0.5 = 0.992$ and $\text{mAP}@0.5:0.95 = 0.853$, substantially improving on the prior YOLOv8-medium baseline. The Temporal Sequence Classifier with a 16-frame buffer achieves macro F1 = 0.96 and overall accuracy of 0.97, with all three classes above F1 = 0.95 on 1,829 validation samples. End-to-end evaluation on full fanning and trophallaxis source datasets confirms that the pipeline correctly identifies the dominant behavior in both scenes by majority vote. Per-bee accuracy on GT-annotated bees ($\text{IoU} \geq 0.15$), restricted to confident predictions, is 75.2% for fanning and 92.6% for trophallaxis — the primary end-to-end classification figures; the conservative lower bound counting uncertain predictions as errors is 67.6% and 88.0% respectively. Scene-level accuracy using naive per-detection scoring against video labels is methodologically unsound due to unlabeled background bees. Tracker robustness in dense and close-contact scenes remains the primary bottleneck for end-to-end system performance.

6. CONCLUSIONS

6.1 Achievement of the Work Objective

The objective of this thesis was to develop and evaluate an automated two-stage pipeline for recognizing individual honeybee behaviors at the hive entrance. This objective has been fully achieved. A complete pipeline was implemented, comprising YOLOv11-medium detection with SAHI tiling, ByteTracker identity persistence, DINOv2-small feature extraction, and a temporal Transformer classifier. The pipeline was trained, evaluated, and demonstrated in an end-to-end inference loop.

All tasks were completed:

- 1- Literature review: Existing bee monitoring methods, YOLO detection variants, ByteTrack, ViT-based features, and behavior recognition approaches were reviewed and compared. The limitations of heuristic behavior inference were identified and documented
- 2- Dataset preparation: Two purpose-built datasets were constructed: a 30,001-image detection dataset and a 104,888-image, 2,969-sequence action recognition dataset spanning three behavior classes
- 3- Detection model: YOLOv11-medium was trained for 238 epochs, achieving $\text{mAP}@0.5 = 0.992$ and $\text{mAP}@0.5:0.95 = 0.853$ on the validation set, improving on the YOLOv8-medium baseline in both metrics.
- 4- Behavior classifier: a temporal Transformer classifier was designed and trained on frozen DINOv2 features (768-dimensional CLS || mean-patch representation) with a comprehensive regularisation suite, achieving macro $F1 = 0.96$ (accuracy = 0.97) on 1,829 validation samples with 16-frame temporal windows.
- 5- End-to-end integration: All pipeline stages were integrated into a single inference loop and evaluated on 20,480 frames, confirming correct functional operation
- 6- Comparison: results were compared against the prior VG TU works, showing improvements in detection quality and introducing a fully learned behavior classifier where only heuristic methods previously existed.

6.2 What Should Be Done Differently

Several aspects of the work could be improved in future iterations:

- Trophallaxis data collection: With only 76 training sequences, trophallaxis classification relies heavily on class weighting. A substantially larger labeled trophallaxis dataset would improve confidence in the generalizability of the $F1 = 0.96$ result
- Consecutive-frame E2E evaluation: The end-to-end pipeline was validated on frames sampled every 6 frames rather than true consecutive video. A proper evaluation on live or continuously recorded footage would more accurately reflect deployment performance and would exercise the temporal model under the conditions it was designed for.
- Tracker robustness: During high-density activity, ByteTracker can lose or switch bee identities, fragmenting the feature buffer. Integration of a re-identification module or an appearance-based tracker would improve the integrity of per-bee sequences in crowded scenes.
- Confidence calibration: The pipeline was trained and evaluated on footage from a single set of recording sessions at one hive entrance. Whether the reported detection and classification performance generalises to different hive setups, camera angles, lighting conditions, or bee subspecies has not been assessed. The in-domain calibration measures (label smoothing, MixUp, R-Drop, entropy regularisation) reduced overconfidence sufficiently that post-hoc temperature scaling found $T \approx 1.0$ on the validation split; whether this calibration holds under domain shift is similarly untested. Establishing both accuracy and calibration robustness across recording conditions would be a prerequisite for deployment in a new beekeeping context.
- Additional behavior classes: behaviors such as washboarding, guarding, and the early stages of swarming were not addressed. Extending the vocabulary to include these would increase the diagnostic value of the system.

6.3 Tool Assessment

The chosen tools met their intended roles well. The Ultralytics framework provided a stable and well-documented training environment for YOLOv11, and the SAHI library integrated without modification. DINOv2 from HuggingFace Transformers provided high-quality frozen features with minimal setup. PyTorch's mixed-precision training and the Supervision library for tracking and

annotation completed the stack without friction. The main practical difficulty was the chunked training strategy required for the detection model a single epoch taking 10–13 minutes made standard long-run training impractical without custom session management. The custom `'chunk_state.yaml'` and `'early_stop_state.yaml'` mechanism resolved this cleanly.

6.4 Summary

This thesis presented an end-to-end two-stage pipeline for individual honey bee behavior recognition at the hive entrance. The pipeline advances the prior VGTU work by replacing heuristic speed-based behavior inference with a supervised temporal Transformer classifier trained on 768-dimensional DINOv2 features (CLS || mean-patch), upgrading detection from YOLOv8 to YOLOv11, and introducing recognition of trophallaxis a social behavior not previously addressed. The system achieves detection $mAP@0.5 = 0.992$ and isolated behavior classification macro $F1 = 0.96$. In end-to-end evaluation on 44,680 source-video frames, the integrated pipeline correctly classified 75.2% of fanning bees and 92.6% of trophallaxis bees among IoU-matched confident predictions demonstrating that fine-grained individual-level bee behavior recognition is achievable with a carefully designed two-stage architecture, proper temporal sequence construction, and limited labeled data.

REFERENCES

1. KLEIN, A. M.; VAISSIÈRE, B. E.; CANE, J. H.; STEFFAN-DEWENTER, I.; CUNNINGHAM, S. A.; KREMEN, C.; TSCHARNTKE, T. 2007. Importance of pollinators in changing landscapes for world crops, *Proceedings of the Royal Society B: Biological Sciences* 274(1608): 303–313.
2. HADJUR, H.; DOREID, A.; LAURENT, L. 2022. Toward an intelligent and efficient beehive: A survey of precision beekeeping systems and services, *Computers and Electronics in Agriculture* 192: 106604.
3. SLEDEVIČ, T.; PLONIS, D. 2023. Toward bee behavioral pattern recognition on hive entrance using YOLOv8. In: 2023 IEEE 10th Jubilee Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE).
4. SLEDEVIČ, T.; ABROMAVIČIUS, V. 2024. Toward bee motion pattern identification on hive landing board. In: IEEE Open Conference of Electrical, Electronic and Information Sciences (eStream).
5. VÁRKONYI, D. T.; SEIXAS Jr., J. L.; HORVÁTH, T. 2023. Dynamic noise filtering for multi-class classification of beehive audio data, *Expert Systems with Applications* 213: 118850.
6. FERRARI, S.; SILVA, M.; GUARINO, M.; BERCKMANS, D. 2008. Monitoring of swarming sounds in bee hives for early detection of the swarming period, *Computers and Electronics in Agriculture* 64(1): 72–77.
7. SHAW, J. A.; NUGENT, W. P.; JOHNSON, J.; BROMENSHENK, J. J.; HENDERSON, B. C.; DEBNAM, S. 2011. Long-wave infrared imaging for non-invasive beehive population assessment, *Optics Express* 19: 399–408.
8. REDMON, J.; DIVVALA, S.; GIRSHICK, R.; FARHADI, A. 2016. You only look once: Unified, real-time object detection. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, p. 779–788.
9. JOCHER, G.; CHAURASIA, A.; QIU, J. 2023. Ultralytics YOLO [Interactive]. Ultralytics [accessed in 2024]. Available from: <<https://doi.org/10.5281/zenodo.7347926>>
10. AKYON, F. C.; ALTINUC, S. O.; TEMIZEL, A. 2022. Slicing aided hyper inference and fine-tuning for small object detection. In: IEEE International Conference on Image Processing (ICIP), Bordeaux, France, p. 966–970.
11. ZHANG, Y.; SUN, P.; JIANG, Y.; YU, D.; WENG, F.; YUAN, Z.; LIAO, P.; LIU, H.; WANG, X. 2022. ByteTrack: Multi-object tracking by associating every detection box. In: European Conference on Computer Vision (ECCV), p. 1–21.
12. DOSOVITSKIY, A.; BEYER, L.; KOLESNIKOV, A.; WEISSENBERN, D.; ZHAI, X.; UNTERTHINER, T.; DEGHANI, M.; MINDERER, M.; HEIGOLD, G.; GELLY, S.; USZKOREIT, J.; HOULSBY, N. 2021. An image is worth 16×16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR).
13. OQUAB, M.; DARCET, T.; MOUTAKANNI, T.; VO, H. V.; SZAFRANIEC, M.; KHALIDOV, V.; FERNANDEZ, P.; HAZIZA, D.; MASSA, F.; EL-NOUBY, A. [et al.] 2024. DINOv2: Learning robust visual features without supervision, *Transactions on Machine Learning Research*: 2304.07193.

14. SIMONYAN, K.; ZISSERMAN, A. 2014. Two-stream convolutional networks for action recognition in videos. In: *Advances in Neural Information Processing Systems 27*, p. 568–576.
15. XIONG, R.; YANG, Y.; HE, D.; ZHENG, K.; ZHENG, S.; XING, C.; ZHANG, H.; LAN, Y.; WANG, L.; LIU, T. 2020. On layer normalization in the Transformer architecture. In: *International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research 119*, p. 10524–10533.
16. SLEDEVIČ, T.; SERACKIS, A.; MATUZEVIČIUS, D.; PLONIS, D.; VDOVIK, G. 2025. Visual recognition of honeybee behavior patterns at the hive entrance, *PLoS One* 20(2): e0318401.
17. SLEDEVIČ, T. 2025. Evaluation of single-shot object detection models for identifying fanning behavior in honeybees at the hive entrance, *Agriculture* 15(15): 1609.
18. VDOVIK, G.; SLEDEVIČ, T. 2025. Temporal encoding strategies for YOLO-based detection of honeybee trophallaxis behavior in precision livestock systems, *Agriculture* 15(22): 2338.
19. VDOVIK, G.; SLEDEVIČ, T.; SERACKIS, A.; PLONIS, D.; MATUZEVIČIUS, D.; ABROMAVIČIUS, V. 2025. Evaluation of deep learning models for insects detection at the hive entrance for a bee behavior recognition system, *Agriculture* 15(10): 1019.
20. KONGSILP, P.; TAETRAGOOL, U.; DUANGPHAKDEE, O. 2024. Individual honey bee tracking in a beehive environment using deep learning and Kalman filter, *Scientific Reports* 14: 969.
21. ROZENBAUM, E.; SHROT, T.; DALTROPHE, H.; KUNYA, Y.; SHAFIR, S. [et al.] 2024. Machine learning-based bee recognition and tracking for advancing insect behavior research, *Artificial Intelligence Review* 57: 245.

APPENDICES

Appendix A Source Code of the Temporal Sequence Classifier

The source code below presents the core implementation of the Temporal Sequence Classifier: the model hyperparameter configuration and the `TemporalSequenceClassifier` class.

Hyperparameter Configuration:

```

1  BUFFER_SIZE = 16
2  MIN_SEQ_LEN = 4
3  VAL_STRIDE = 8
4
5  FEATURE_DIM = 768
6  NUM_CLASSES = 3
7  NUM_HEADS = 6
8  NUM_LAYERS = 3
9  DROPOUT = 0.35
10
11 BATCH_SIZE = 64
12 NUM_WORKERS = 0
13 MAX_EPOCHS = 150
14 PATIENCE = 25
15 LR = 1e-4
16 WEIGHT_DECAY = 0.1
17 WARMUP_EPOCHS = 10
18 LABEL_SMOOTHING = 0.1
19
20 NOISE_STD = 0.05
21 FRAME_DROP_MAX = 3
22 JITTER_MAX = 3
23 MIXUP_PROB = 0.50
24 MIXUP_ALPHA = 0.60
25 ENTROPY_WEIGHT = 0.05
26 RDROP_WEIGHT = 0.20

```

Model definition:

```

1  class TemporalSequenceClassifier(nn.Module):
2      def __init__(self, feature_dim=768, num_classes=3, num_heads=6,
3                  num_layers=3, dropout=0.35, max_seq_len=17):
4          super().__init__()
5          self.cls_token = nn.Parameter(torch.randn(1, 1, feature_dim) * 0.02)
6          self.pos_embed = nn.Embedding(max_seq_len, feature_dim)
7          encoder_layer = nn.TransformerEncoderLayer(
8              d_model=feature_dim, nhead=num_heads,
9              dim_feedforward=feature_dim * 2,
10             dropout=dropout, batch_first=True, norm_first=True,
11         )
12         self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
13         self.norm = nn.LayerNorm(feature_dim)
14         self.classifier = nn.Sequential(
15             nn.Linear(feature_dim, 256), nn.GELU(),
16             nn.Dropout(dropout), nn.Linear(256, num_classes),
17         )
18         self._init_weights()
19
20     def _init_weights(self):
21         nn.init.trunc_normal_(self.cls_token, std=0.02)
22         nn.init.normal_(self.pos_embed.weight, std=0.02)
23         for m in self.modules():
24             if isinstance(m, nn.Linear):
25                 nn.init.trunc_normal_(m.weight, std=0.02)
26                 if m.bias is not None:
27                     nn.init.zeros_(m.bias)
28
29     def forward(self, x, padding_mask=None):
30         B, T, _ = x.shape
31         cls = self.cls_token.expand(B, -1, -1)
32         x = torch.cat([cls, x], dim=1)
33         pos = torch.arange(T + 1, device=x.device).unsqueeze(0)
34         x = x + self.pos_embed(pos)
35         if padding_mask is not None:
36             cls_mask = torch.zeros(B, 1, dtype=torch.bool, device=x.device)
37             full_mask = torch.cat([cls_mask, padding_mask], dim=1)
38         else:
39             full_mask = None
40         x = self.transformer(x, src_key_padding_mask=full_mask)
41         return self.classifier(self.norm(x[:, 0]))

```

Figure A.1 - Temporal Sequence Classifier training curves

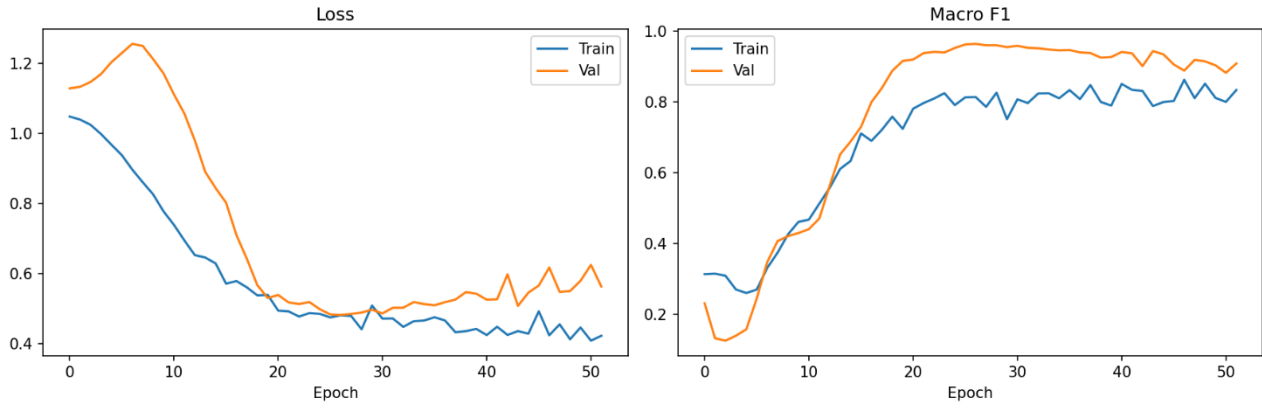


Figure A.1 - Temporal Sequence Classifier training curves